01_Case study: E-commerce Marketing and sales

January 13, 2026

1 Case study: E-commerce Marketing and sales

1.1 Unziping files

[ ]:
```
!unzip ecom.zip
```

```
unzip:  cannot find or open ecom.zip, ecom.zip.zip or ecom.zip.ZIP.
```

2 File we have:

- Online_sales.csv
- CustomersData.xlsx
- Discount_Coupon.csv
- Marketing_spend.csv
- Tax_amount.xlsx

#1] Load and Inspect Data - in this we will import liberies pandas and numpy - we will import all the files

[ ]:
```
#Importing Liberies
import pandas as pd
import numpy as np
```

2.1 i)Discount_Coupon.csv

[ ]:
```
discount_coupon = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Business␣
↪Case study/01/Discount_Coupon.csv')
discount_coupon.head()
```

[ ]:

	Month	Product_Category	Coupon_Code	Discount_pct
0	Jan	Apparel	SALE10	10
1	Feb	Apparel	SALE20	20
2	Mar	Apparel	SALE30	30
3	Jan	Nest-USA	ELEC10	10
4	Feb	Nest-USA	ELEC20	20

2.2 ii)Marketing_Spend.csv

[ ]:
```
marketing_spend = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Business␣
↪Case study/01/Marketing_Spend.csv')
marketing_spend.head()
```

[ ]:

	Date	Offline_Spend	Online_Spend
0	1/1/2019	4500	2424.50
1	1/2/2019	4500	3480.36
2	1/3/2019	4500	1576.38
3	1/4/2019	4500	2928.55
4	1/5/2019	4500	4055.30

2.3 iii) Tax_amount.xlsx

[ ]:
```
taxAmount = pd.read_excel('/content/drive/MyDrive/Colab Notebooks/Business Case␣
↪study/01/Tax_amount.xlsx')
taxAmount.head()
```

[ ]:

	Product_Category	GST
0	Nest-USA	0.10
1	Office	0.10
2	Apparel	0.18
3	Bags	0.18
4	Drinkware	0.18

2.4 iv) Customers.csv

---

1. Columns
   - CustomerId : Has Customer Id it as number format int64 all CustomerId are unique
   - Gender : it contains M or F which tell if customer is male or female
   - Location: it contain city name in location
   - Tenure_Months: it is int64(integer) no-negative vaule it tells How long the customer has been associated with the company
2. is null vaule present: no null values present
3. what the shape
   - Rows: 1468
   - Columns: 4

---

1. From this Dataset What do I understand?
   - from this dataset i understand that:
     - Customer location,
     - Gender if male or female

– how many months the customer is associated with company

[ ]:
```
customers = pd.read_excel("/content/drive/MyDrive/Colab Notebooks/Business Case␣
↪study/01/CustomersData.xlsx")
customers.tail()
```

[ ]:

	CustomerID	Gender	Location	Tenure_Months
1463	14438	F	New York	41
1464	12956	F	Chicago	48
1465	15781	M	New Jersey	19
1466	14410	F	New York	45
1467	14600	F	California	7

[ ]:
```
# Info of Customers
customers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1468 entries, 0 to 1467
Data columns (total 4 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     1468 non-null   int64
 1   Gender         1468 non-null   object
 2   Location       1468 non-null   object
 3   Tenure_Months  1468 non-null   int64
dtypes: int64(2), object(2)
memory usage: 46.0+ KB
```

[ ]:
```
# check if there is null values or not
customers.isna()
```

[ ]:

	CustomerID	Gender	Location	Tenure_Months
0	False	False	False	False
1	False	False	False	False
2	False	False	False	False
3	False	False	False	False
4	False	False	False	False
…	…	…	…	…
1463	False	False	False	False
1464	False	False	False	False
1465	False	False	False	False
1466	False	False	False	False
1467	False	False	False	False

[1468 rows x 4 columns]

[ ]:
```
# Shape
customers.shape
```

[ ]: (1468, 4)

[ ]:
```
# is customer is unique
customers['CustomerID'].nunique()
```

[ ]: 1468

2.5 (v) Online_Sales.csv

---

1. Column:
   - CustomerId:
     - Unique identifier for each customer.
     - A customer can appear multiple times because the same customer may place multiple orders over time.
   - Transcation_Id:
     - Unique identifier for each transaction/order.
     - It represents a purchase event and is not null.
     - A single customer can have multiple transaction IDs.
   - Transcation_Date:
     - A single customer can have multiple transaction IDs.
     - Initially stored as an **object/string**, it has been converted to datetime format to enable time-based analysis such as monthly trends, retention, and cohort analysis.
   - Product_SKU:
     - Stock Keeping Unit (SKU) that uniquely identifies a product.
     - Useful for product-level performance analysis.
   - Product_Description:
     - String Description of Product purchased
     - Helps in understanding product details
   - Product_Category:
     - Category to which the product belongs (e.g., Electronics, Clothing, etc.).
     - help full to check Category operation
   - Quantity:
     - Number of units of a product purchased in a transaction. -This is a integer column and directly affects `Avg_price` calculations.
   - Avg_Price:
     - Average price per unit of the product.
     - Used along with `quantity` to calculate the base transaction value.
   - Delivery_Charges:
     - Additional charges applied for delivering the product.
     - These charges contribute to the final invoice value.
   - Coupon_Status:
     - Indicates whether a discount coupon was applied to the transaction or not.
     - Useful for analyzing the impact of discounts on sales and revenue.
2. Shape:
   - Rows: 52924
   - column: 10

[ ]:
```
online_sales = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Business␣
↪Case study/01/Online_Sales.csv')
online_sales.head()
```

[ ]:
```
   CustomerID  Transaction_ID Transaction_Date     Product_SKU  \
0       17850           16679         1/1/2019  GGOENEBJ079499
1       17850           16680         1/1/2019  GGOENEBJ079499
2       17850           16681         1/1/2019  GGOEGFKQ020399
3       17850           16682         1/1/2019  GGOEGAAB010516
4       17850           16682         1/1/2019  GGOEGBJL013999

                                Product_Description Product_Category  \
0  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
1  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
2             Google Laptop and Cell Phone Stickers           Office
3  Google Men's 100% Cotton Short Sleeve Hero Tee…          Apparel
4                    Google Canvas Tote Natural/Navy             Bags

   Quantity  Avg_Price  Delivery_Charges Coupon_Status
0         1     153.71               6.5          Used
1         1     153.71               6.5          Used
2         1       2.05               6.5          Used
3         5      17.53               6.5      Not Used
4         1      16.50               6.5          Used
```

[ ]:
```
#check shape
online_sales.shape
```

[ ]: (52924, 10)

[ ]:
```
#Info
online_sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52924 entries, 0 to 52923
Data columns (total 10 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   CustomerID           52924 non-null  int64
 1   Transaction_ID       52924 non-null  int64
 2   Transaction_Date     52924 non-null  object
 3   Product_SKU          52924 non-null  object
 4   Product_Description  52924 non-null  object
 5   Product_Category     52924 non-null  object
 6   Quantity             52924 non-null  int64
 7   Avg_Price            52924 non-null  float64
 8   Delivery_Charges     52924 non-null  float64
```

```
 9   Coupon_Status        52924 non-null  object
dtypes: float64(2), int64(3), object(5)
memory usage: 4.0+ MB
```

[ ]:
```python
#Check Null
online_sales.isna()
```

[ ]:
```
       CustomerID  Transaction_ID  Transaction_Date  Product_SKU  \
0           False           False             False        False
1           False           False             False        False
2           False           False             False        False
3           False           False             False        False
4           False           False             False        False
…           …              …                …           …
52919       False           False             False        False
52920       False           False             False        False
52921       False           False             False        False
52922       False           False             False        False
52923       False           False             False        False

       Product_Description  Product_Category  Quantity  Avg_Price  \
0                    False             False     False      False
1                    False             False     False      False
2                    False             False     False      False
3                    False             False     False      False
4                    False             False     False      False
…                    …                …        …         …
52919                False             False     False      False
52920                False             False     False      False
52921                False             False     False      False
52922                False             False     False      False
52923                False             False     False      False

       Delivery_Charges  Coupon_Status
0                 False          False
1                 False          False
2                 False          False
3                 False          False
4                 False          False
…                 …             …
52919             False          False
52920             False          False
52921             False          False
52922             False          False
52923             False          False

[52924 rows x 10 columns]
```

2.5.1 a) Correcting dataframe Transcation_Date from object to datetime

[ ]:
```
online_sales['Transaction_Date'] = pd.
 ↪to_datetime(online_sales['Transaction_Date'])
online_sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52924 entries, 0 to 52923
Data columns (total 10 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   CustomerID           52924 non-null  int64
 1   Transaction_ID       52924 non-null  int64
 2   Transaction_Date     52924 non-null  datetime64[ns]
 3   Product_SKU          52924 non-null  object
 4   Product_Description  52924 non-null  object
 5   Product_Category     52924 non-null  object
 6   Quantity             52924 non-null  int64
 7   Avg_Price            52924 non-null  float64
 8   Delivery_Charges     52924 non-null  float64
 9   Coupon_Status        52924 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(3), object(4)
memory usage: 4.0+ MB
```

2.5.2 b) Creating Month column to enable time-based analysis using groupby

[ ]:
```
online_sales['Transaction_month'] = online_sales['Transaction_Date'].dt.month
online_sales.head(10)
```

[ ]:
```
   CustomerID  Transaction_ID Transaction_Date     Product_SKU  \
0       17850           16679       2019-01-01  GGOENEBJ079499
1       17850           16680       2019-01-01  GGOENEBJ079499
2       17850           16681       2019-01-01  GGOEGFKQ020399
3       17850           16682       2019-01-01  GGOEGAAB010516
4       17850           16682       2019-01-01  GGOEGBJL013999
5       17850           16682       2019-01-01  GGOEGBMJ013399
6       17850           16682       2019-01-01  GGOEGDHC018299
7       17850           16682       2019-01-01  GGOEGDHG014499
8       17850           16682       2019-01-01  GGOEGDWC020199
9       13047           16682       2019-01-01  GGOEGGOA017399

                                Product_Description Product_Category  \
0  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
1  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
2             Google Laptop and Cell Phone Stickers           Office
3  Google Men's 100% Cotton Short Sleeve Hero Tee…          Apparel
4                    Google Canvas Tote Natural/Navy             Bags
```

```
5                                        Sport Bag             Bags
6                          Google 22 oz Water Bottle        Drinkware
7                     Google Infuser-Top Water Bottle        Drinkware
8                         Engraved Ceramic Google Mug        Drinkware
9                                           Maze Pen           Office

   Quantity  Avg_Price  Delivery_Charges Coupon_Status  Transaction_month
0         1     153.71               6.5          Used                  1
1         1     153.71               6.5          Used                  1
2         1       2.05               6.5          Used                  1
3         5      17.53               6.5      Not Used                  1
4         1      16.50               6.5          Used                  1
5        15       5.15               6.5          Used                  1
6        15       3.08               6.5      Not Used                  1
7        15      10.31               6.5       Clicked                  1
8         5       9.27               6.5          Used                  1
9        52       0.98               6.5          Used                  1
```

[ ]:
```
#checking if it's sorted by transcation_date
online_sales['Transaction_Date'].is_monotonic_increasing
```

[ ]:
```
True
```

3 Business Question

3.1 Q1: Identify the months with the highest and lowest customer acquisition count. What strategies could be implemented to address the fluctuations and ensure consistent growth throughout the year?

[ ]:
```
# Finding the first Transaction Date
acquisition_df = online_sales.groupby('CustomerID')['Transaction_Date'].min()
```

[ ]:
```
#Reset Index
acquisition_df= acquisition_df.reset_index()
```

[ ]:
```
# Rename output column for readability
acquisition_df.rename(columns={'Transaction_Date':
 ↪'First_Transaction_Date'},inplace=True)
acquisition_df.head()
```

[ ]:
```
   CustomerID First_Transaction_Date
0       12346             2019-09-15
1       12347             2019-03-24
2       12348             2019-06-22
3       12350             2019-12-14
4       12356             2019-09-15
```

[ ]:
```
#need to get acquisition per month
acquisition_df['acquisition_month'] = acquisition_df['First_Transaction_Date'].
↪dt.month
acquisition_df.head()
```

[ ]:

	CustomerID	First_Transaction_Date	acquisition_month
0	12346	2019-09-15	9
1	12347	2019-03-24	3
2	12348	2019-06-22	6
3	12350	2019-12-14	12
4	12356	2019-09-15	9

[ ]:
```
#get per month acquisition
acquisition_per_month = acquisition_df.
↪groupby('acquisition_month')['CustomerID'].count()
```

[ ]:
```
#Reset Index
acquisition_per_month = acquisition_per_month.reset_index()
```

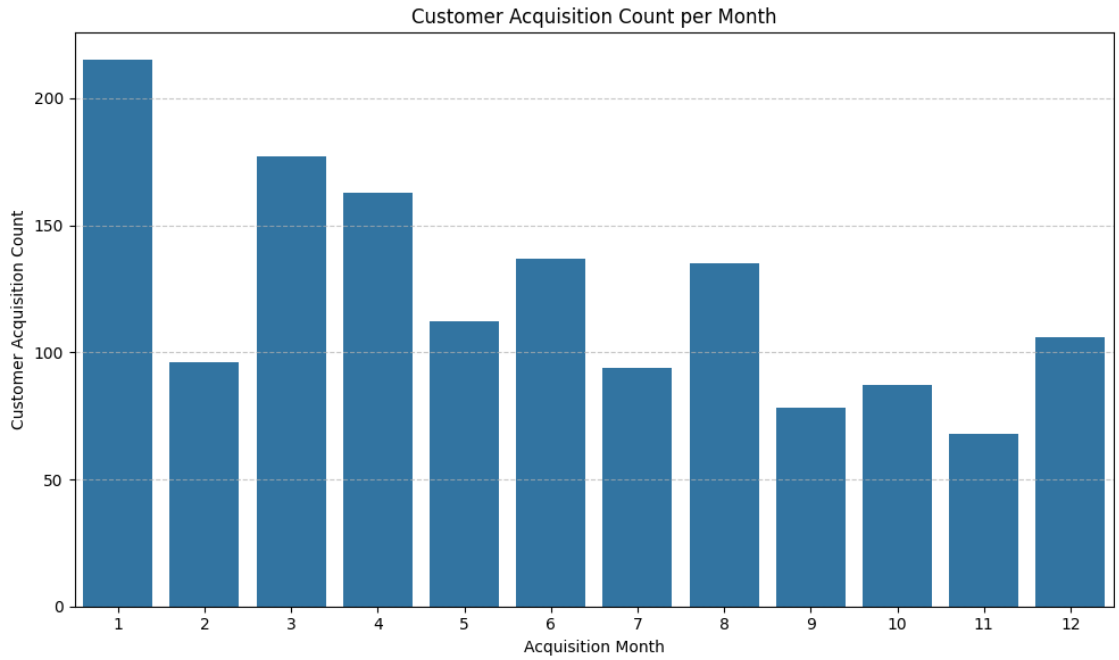
[ ]:
```
#Rename output column for readability
acquisition_per_month.rename(columns={'CustomerID':
↪'customer_acquisition_count'}, inplace=True)
```

[ ]:
```
acquisition_per_month.describe()
```

[ ]:

	acquisition_month	customer_acquisition_count
count	12.000000	12.000000
mean	6.500000	122.333333
std	3.605551	44.354631
min	1.000000	68.000000
25%	3.750000	92.250000
50%	6.500000	109.000000
75%	9.250000	143.500000
max	12.000000	215.000000

[ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.barplot(x='acquisition_month', y='customer_acquisition_count',␣
↪data=acquisition_per_month)
plt.title('Customer Acquisition Count per Month')
plt.xlabel('Acquisition Month')
plt.ylabel('Customer Acquisition Count')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



- to get higesht customer acquisition count

```
[ ]: acquisition_per_month['customer_acquisition_count'].max()
```

```
[ ]: 215
```

- to get lowest customer acquisition count

```
[ ]: acquisition_per_month['customer_acquisition_count'].min()
```

```
[ ]: 68
```

Q1 Solution:

3.1.1 Methodology

- Customer acquisition is defined as the **first purchase made by a customer**.
- The earliest transaction date was identified for each unique customer.
- Customers were grouped by the month of their first transaction to calculate monthly acquisition counts.

3.1.2 Key Findings

- January recorded the **highest customer acquisition** (215 customers).
- November recorded the **lowest customer acquisition** (68 customers).

3.1.3 Visual Analysis

- A monthly bar chart highlights strong acquisition in early months and a gradual decline toward year-end.

3.1.4 Business Interpretation

- Higher acquisition in January may be driven by New Year demand, promotions, and renewed customer intent.
- Lower acquisition in November suggests seasonal slowdown or reduced marketing activity.

3.1.5 Recommendations

- Replicate successful January campaigns during weaker months.
- Introduce targeted promotions and referral programs in low-acquisition periods.

3.2 Q2: Analyze the data to determine if certain months consistently show higher or lower acquisition. How can the company capitalize on high-performing months and improve performance during slower periods?

i) Observations:

- Monthly customer acquisition shows a seasonal trend, not random ups and downs.
- The strongest onboarding happens in the early part of the year.
- January, March, April repeatedly record higher customer acquisition.
- Acquisition tends to taper off as the year progresses.
- September, October, and November consistently fall among the weakest months for new customer additions.
- Overall, demand and acquisition performance appear strongest at the start of the year and softer toward year-end.

---

ii) Business Interpretation

- Higher customer acquisition in the early months may be driven by New Year momentum, as customers are more inclined to explore new products or services, influenced by renewed goals, fresh budgets, and increased purchase intent at the start of the year.
- Early-year acquisition spikes can be attributed to stronger marketing pushes, higher advertising budgets, and targeted promotional campaigns typically planned for Q1.
- Additionally, renewed customer intent at the start of the year driven by fresh goals, budgets, and requirements may encourage customers to explore new products or services.
- The decline observed in later months likely reflects a seasonal slowdown, reduced campaign intensity, and shifting business priorities toward year end.
- Customer fatigue from repeated promotions and delayed purchasing decisions ahead of major year end sales events may further contribute to lower acquisition during this period.
- Overall, the data suggests a repeatable seasonal acquisition cycle that can be strategically leveraged to better time campaigns, allocate marketing budgets, and plan sales efforts.

---

iii) Recommendations

- **Capitalizing on high- performance months:**
  - Increase marketing investment and campaign intensity during historically strong acquisition months.
  - Launch new products, referral programs, or limited-time offers when customer intent is already high.
  - Strengthen onboarding and retention efforts to maximize lifetime value of newly acquired customers.
- **Improving performance during slower months:**
  - Introduce targeted promotions and personalized campaigns to stimulate demand.
  - Reallocate marketing budgets toward low-performing months based on historical trends.
  - Run reactivation campaigns for inactive customers and bundle offers to increase purchase motivation. ____ #### iv) Final verdict:
- Customer acquisition follows a seasonal trend, with strong performance early in the year and weaker results toward the end. By aligning marketing strategies with these patterns, the company can stabilize acquisition levels and drive more consistent growth throughout the year.

[ ]:
```
```

3.3 Q3: Identify periods with the strongest and weakest retention rates. What strategies could be implemented to improve retention during weaker months?

[ ]:
```
#Retention formula
# Retention (Month X) = customers in X who return in X+1 / customers in X
```

[ ]:
```
online_sales[['CustomerID', 'Transaction_month']].head()
```

[ ]:
```
   CustomerID  Transaction_month
0       17850                  1
1       17850                  1
2       17850                  1
3       17850                  1
4       17850                  1
```

[ ]:
```
online_sales[['CustomerID', 'Transaction_month']]
```

[ ]:
```
        CustomerID  Transaction_month
0            17850                  1
1            17850                  1
2            17850                  1
3            17850                  1
4            17850                  1
…            …                    …
52919        14410                 12
52920        14410                 12
```

```
52921       14410                 12
52922       14600                 12
52923       14600                 12

[52924 rows x 2 columns]
```

[ ]:
```
online_sales.head()
```

[ ]:
```
   CustomerID  Transaction_ID Transaction_Date     Product_SKU  \
0       17850           16679       2019-01-01  GGOENEBJ079499
1       17850           16680       2019-01-01  GGOENEBJ079499
2       17850           16681       2019-01-01  GGOEGFKQ020399
3       17850           16682       2019-01-01  GGOEGAAB010516
4       17850           16682       2019-01-01  GGOEGBJL013999

                                Product_Description Product_Category  \
0  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
1  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
2             Google Laptop and Cell Phone Stickers           Office
3  Google Men's 100% Cotton Short Sleeve Hero Tee…          Apparel
4                    Google Canvas Tote Natural/Navy             Bags

   Quantity  Avg_Price  Delivery_Charges Coupon_Status  Transaction_month
0         1     153.71               6.5          Used                  1
1         1     153.71               6.5          Used                  1
2         1       2.05               6.5          Used                  1
3         5      17.53               6.5      Not Used                  1
4         1      16.50               6.5          Used                  1
```

[ ]:
```
#Creating a Retention base table where we will only have unique CustomerID with␣
↪Transaction_month no duplicate
retention_df = online_sales[['CustomerID', 'Transaction_month']].
↪drop_duplicates()
retention_df.head()
```

[ ]:
```
    CustomerID  Transaction_month
0        17850                  1
9        13047                  1
26       12583                  1
46       13748                  1
65       15100                  1
```

[ ]:
```
#Build customer sets per month(creating set of customer purchased in each month)
monthly_customers =(retention_df.groupby('Transaction_month')['CustomerID'].
↪apply(set))
monthly_customers.head()
```

[ ]:
```
Transaction_month
1    {17920, 14849, 15363, 17924, 17925, 17412, 174…
2    {13831, 15880, 16905, 16393, 15373, 13838, 158…
3    {17920, 16898, 14850, 13319, 12808, 14344, 158…
4    {16385, 16898, 17411, 13319, 15881, 16907, 133…
5    {15361, 16899, 16393, 15881, 13324, 17404, 179…
Name: CustomerID, dtype: object
```

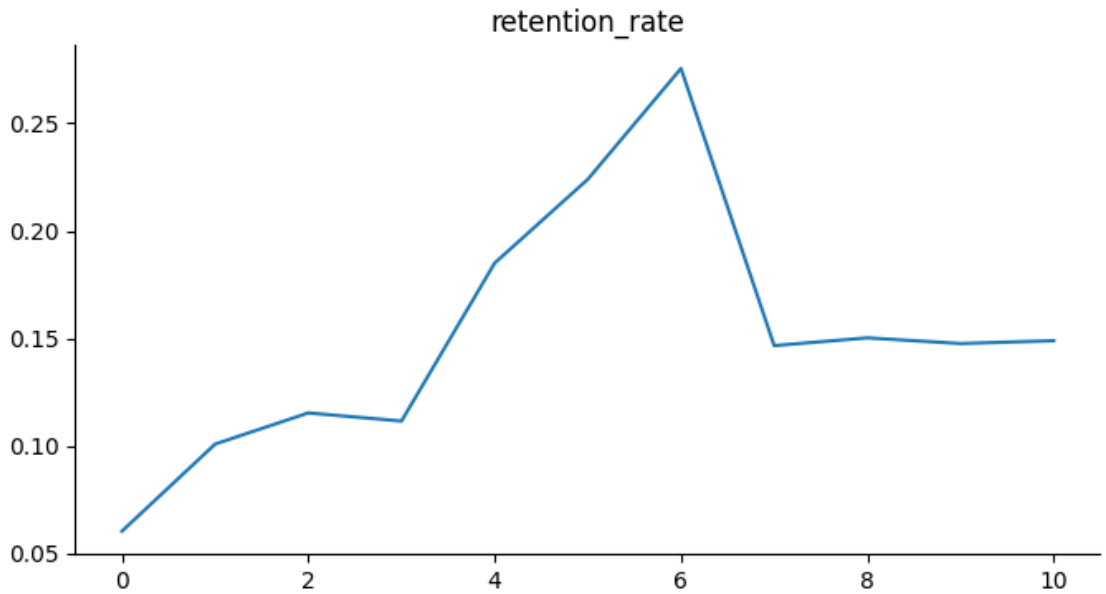
[ ]:
```
#Calculate retention month by month

retention_results = []

for month in range(1,12):
  current_month_customers = monthly_customers.get(month, set())
  next_month_customers = monthly_customers.get(month +1, set())

  if len(current_month_customers)== 0:
    retention_rate = 0
  else:
    retained =current_month_customers & next_month_customers
    retention_rate = len(retained) / len(current_month_customers)

  retention_results.append({'month':month,
                            'retention_rate':retention_rate})
```

[ ]:
```
#Convert to Dataframe
retention_rate_df = pd.DataFrame(retention_results)
retention_rate_df
```

[ ]:

	month	retention_rate
0	1	0.060465
1	2	0.100917
2	3	0.115385
3	4	0.111607
4	5	0.185000
5	6	0.223938
6	7	0.275424
7	8	0.146667
8	9	0.150259
9	10	0.147619
10	11	0.148936

[ ]:
```
from matplotlib import pyplot as plt
retention_rate_df['retention_rate'].plot(kind='line', figsize=(8, 4),␣
 ↪title='retention_rate')
plt.gca().spines[['top', 'right']].set_visible(False)
```



[ ]:
```
# Identify Strongest & weakest retention months
best_month = retention_rate_df.loc[retention_rate_df['retention_rate'].idxmax()]
worst_month = retention_rate_df.loc[retention_rate_df['retention_rate'].
 ↪idxmin()]

best_month, worst_month
```

[ ]:
```
(month             7.000000
 retention_rate    0.275424
 Name: 6, dtype: float64,
 month             1.000000
 retention_rate    0.060465
 Name: 0, dtype: float64)
```

i) Observations

- Retention rates show meaningful month-to-month variation, indicating changing customer engagement levels.
- Certain months show strong customer continuity into the following month.
-

3.4 Some months experience sharp drop-offs, indicating higher churn.

ii) Business Interpretation

- Strong retention months may align with:
  - Better post-purchase engagement

  - Ongoing promotions
  - High product satisfaction
- Weak retention months may indicate:
  - Lack of follow-up communication
  - One-time promotional purchases
  -

3.5 Reduced perceived value after first purchase

- July shows the strongest customer retention, indicating higher repeat engagement during this period.
- January exhibits the weakest retention, suggesting higher churn after initial purchases.

iv) Final Verdict Retention shows clear month-to-month variation, indicating that customer continuity depends heavily on engagement strategy rather than acquisition volume alone. By strengthening post-purchase touchpoints and targeted re-engagement during weak months, the company can significantly improve overall customer lifetime value.

3.6 Q4: Analyze customer behavior during high-retention months and suggest ways to replicate this success throughout the year.

- High-Retention Month Identified: **July** shows the strongest customer retention based on month-to-month return analysis (from Q3).

[ ]:
```
july_df = online_sales[online_sales['Transaction_month'] == 7]
july_df.head()
```

[ ]:
```
       CustomerID  Transaction_ID Transaction_Date     Product_SKU  \
24608       17841           31689       2019-07-01  GGOENEBJ079499
24609       17841           31691       2019-07-01  GGOENEBB078899
24610       17841           31691       2019-07-01  GGOENEBQ078999
24611       17841           31692       2019-07-01  GGOENEBJ079499
24612       17841           31693       2019-07-01  GGOEGHPB071610

                                   Product_Description Product_Category  \
24608  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
24609              Nest Cam Indoor Security Camera - USA         Nest-USA
24610             Nest Cam Outdoor Security Camera - USA         Nest-USA
24611  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
24612                                  Google Twill Cap          Apparel

       Quantity  Avg_Price  Delivery_Charges Coupon_Status  Transaction_month
24608         2     149.00               6.0       Clicked                  7
24609         1     119.00               6.0          Used                  7
24610         2     119.00               6.0      Not Used                  7
24611         1     149.00               6.0       Clicked                  7
24612         1      10.99               6.0       Clicked                  7
```

[ ]:
```
# we need to find total count of coupons used checked or not used
july_df['Coupon_Status'].value_counts()
```

[ ]:
```
Coupon_Status
Clicked     2705
Used        1757
Not Used     789
Name: count, dtype: int64
```

[ ]:
```
# getting info of discountCoupon
discount_coupon.head(50)
```

[ ]:

	Month	Product_Category	Coupon_Code	Discount_pct
0	Jan	Apparel	SALE10	10
1	Feb	Apparel	SALE20	20
2	Mar	Apparel	SALE30	30
3	Jan	Nest-USA	ELEC10	10
4	Feb	Nest-USA	ELEC20	20
5	Mar	Nest-USA	ELEC30	30
6	Jan	Office	OFF10	10
7	Feb	Office	OFF20	20
8	Mar	Office	OFF30	30
9	Jan	Drinkware	EXTRA10	10
10	Feb	Drinkware	EXTRA20	20
11	Mar	Drinkware	EXTRA30	30
12	Jan	Lifestyle	EXTRA10	10
13	Feb	Lifestyle	EXTRA20	20
14	Mar	Lifestyle	EXTRA30	30
15	Jan	Bags	AIO10	10
16	Feb	Bags	AIO20	20
17	Mar	Bags	AIO30	30
18	Jan	Notebooks	NOTES10	10
19	Feb	Notebooks	NOTES20	20
20	Mar	Notebooks	NOTES30	30
21	Jan	Headgear	HGEAR10	10
22	Feb	Headgear	HGEAR20	20
23	Mar	Headgear	HGEAR30	30
24	Jan	Nest	NE10	10
25	Feb	Nest	NE20	20
26	Mar	Nest	NE30	30
27	Jan	Waze	WEMP10	10
28	Feb	Waze	WEMP20	20
29	Mar	Waze	WEMP30	30
30	Jan	Bottles	BT10	10
31	Feb	Bottles	BT20	20
32	Mar	Bottles	BT30	30
33	Jan	Nest-Canada	NCA10	10

```
34   Feb           Nest-Canada       NCA20            20
35   Mar           Nest-Canada       NCA30            30
36   Jan            Housewares       HOU10            10
37   Feb            Housewares       HOU20            20
38   Mar            Housewares       HOU30            30
39   Jan           Accessories       ACC10            10
40   Feb           Accessories       ACC20            20
41   Mar           Accessories       ACC30            30
42   Jan            Gift Cards        GC10            10
43   Feb            Gift Cards        GC20            20
44   Mar            Gift Cards        GC30            30
45   Jan  Notebooks & Journals        NJ10            10
46   Feb  Notebooks & Journals        NJ20            20
47   Mar  Notebooks & Journals        NJ30            30
48   Jan                Android       AND10            10
49   Feb                Android       AND20            20
```

[ ]:
```python
discount_coupon.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 4 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Month             204 non-null    object
 1   Product_Category  204 non-null    object
 2   Coupon_Code       204 non-null    object
 3   Discount_pct      204 non-null    int64
dtypes: int64(1), object(3)
memory usage: 6.5+ KB
```

[ ]:
```python
# How many columns are there in discount_Coupon
discount_coupon.columns
```

[ ]:
```
Index(['Month', 'Product_Category', 'Coupon_Code', 'Discount_pct'],
dtype='object')
```

[ ]:
```python
#Now here is a issue the Month is in String/Object we need month in number so␣
↪we can merge it
#Month name to number mapping
month_map = {
    'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4,
    'May': 5, 'Jun': 6, 'Jul': 7, 'Aug': 8,
    'Sep': 9, 'Oct': 10, 'Nov': 11, 'Dec': 12
}

discount_coupon['Transaction_month'] = discount_coupon['Month'].map(month_map)
```

```
discount_coupon.head()
```

[ ]:

	Month	Product_Category	Coupon_Code	Discount_pct	Transaction_month
0	Jan	Apparel	SALE10	10	1
1	Feb	Apparel	SALE20	20	2
2	Mar	Apparel	SALE30	30	3
3	Jan	Nest-USA	ELEC10	10	1
4	Feb	Nest-USA	ELEC20	20	2

[ ]:
```
# finding July month discount
discount_july = discount_coupon[discount_coupon['Transaction_month'] == 7]
discount_july.head()
```

[ ]:

	Month	Product_Category	Coupon_Code	Discount_pct	Transaction_month
102	Jul	Apparel	SALE10	10	7
105	Jul	Nest-USA	ELEC10	10	7
108	Jul	Office	OFF10	10	7
111	Jul	Drinkware	EXTRA10	10	7
114	Jul	Lifestyle	EXTRA10	10	7

[ ]:
```
july_df.head()
```

[ ]:

	CustomerID	Transaction_ID	Transaction_Date	Product_SKU	\
24608	17841	31689	2019-07-01	GGOENEBJ079499	
24609	17841	31691	2019-07-01	GGOENEBB078899	
24610	17841	31691	2019-07-01	GGOENEBQ078999	
24611	17841	31692	2019-07-01	GGOENEBJ079499	
24612	17841	31693	2019-07-01	GGOEGHPB071610	

	Product_Description	Product_Category	\
24608	Nest Learning Thermostat 3rd Gen-USA - Stainle…	Nest-USA	
24609	Nest Cam Indoor Security Camera - USA	Nest-USA	
24610	Nest Cam Outdoor Security Camera - USA	Nest-USA	
24611	Nest Learning Thermostat 3rd Gen-USA - Stainle…	Nest-USA	
24612	Google Twill Cap	Apparel	

	Quantity	Avg_Price	Delivery_Charges	Coupon_Status	Transaction_month
24608	2	149.00	6.0	Clicked	7
24609	1	119.00	6.0	Used	7
24610	2	119.00	6.0	Not Used	7
24611	1	149.00	6.0	Clicked	7
24612	1	10.99	6.0	Clicked	7

[ ]:
```
#Merge the July_Sales and July Discount
july_merge_df = july_df.merge(discount_july, on=['Transaction_month',␣
↪'Product_Category'], how='left')
july_merge_df.tail()
```

[ ]:
```
      CustomerID  Transaction_ID Transaction_Date     Product_SKU  \
5246       13081           34269       2019-07-31  GGOEGDWR015799
5247       13081           34270       2019-07-31  GGOENEBB078899
5248       13081           34270       2019-07-31  GGOENEBQ078999
5249       13081           34272       2019-07-31  GGOEGAAJ032616
5250       13081           34272       2019-07-31  GGOEYAAB031816

                              Product_Description Product_Category  Quantity  \
5246                          Red Shine 15 oz Mug        Drinkware         1
5247         Nest Cam Indoor Security Camera - USA         Nest-USA         2
5248        Nest Cam Outdoor Security Camera - USA         Nest-USA         1
5249  Google Men's Short Sleeve Badge Tee Charcoal          Apparel         1
5250     YouTube Men's Short Sleeve Hero Tee Black          Apparel         1

      Avg_Price  Delivery_Charges Coupon_Status  Transaction_month Month  \
5246      10.39               6.0          Used                  7   Jul
5247     119.00               6.0       Clicked                  7   Jul
5248     119.00               6.0       Clicked                  7   Jul
5249      13.29               6.0          Used                  7   Jul
5250      16.99               6.0       Clicked                  7   Jul

     Coupon_Code  Discount_pct
5246     EXTRA10          10.0
5247      ELEC10          10.0
5248      ELEC10          10.0
5249      SALE10          10.0
5250      SALE10          10.0
```

[ ]:
```python
# Coupon usage behavior
july_merge_df['Coupon_Status'].value_counts()
```

[ ]:
```
Coupon_Status
Clicked     2705
Used        1757
Not Used     789
Name: count, dtype: int64
```

[ ]:
```python
# Discount percentage distribution
july_merge_df['Discount_pct'].value_counts(dropna=False)
```

[ ]:
```
Discount_pct
10.0    5196
NaN       55
Name: count, dtype: int64
```

[ ]:
```python
#Product category popularity
july_merge_df['Product_Category'].value_counts()
```

[ ]:
```
Product_Category
Apparel                 2182
Nest-USA                1161
Office                   549
Lifestyle                383
Drinkware                298
Bags                     216
Notebooks & Journals     150
Headgear                 119
Bottles                   40
Fun                       35
Nest-Canada               27
Waze                      25
Housewares                22
Google                    12
Gift Cards                10
Backpacks                  8
Android                    8
Accessories                6
Name: count, dtype: int64
```

[ ]:
```python
# Average order value
print(july_merge_df['Avg_Price'].mean())
```

```
38.07831460674158
```

3.6.1 i) Observations:

- From July transaction and coupon analysis:
  - Majority of customers clicked or used coupons in July.
- Discount structure is consistent
  - Most applied discounts are ~10%, indicating modest but effective incentives.
- Product category concentration:
  - Higher Repeat purchases observed in:
    - Nest-USA
    - Apparel
    - Office
- Stable Delivery charges:
  - Delivery fees remain constant, reducing post-purchase friction
- Repeat-Friendly products
  - Many products are ecosystem-based (smart devices, accessories), encouraging follow-up purchases.

3.6.2 iii) Strategies to use This Success Across the Year

- Standardize **July-style** retention playbook
  - Introduce moderate, always-on discounts (5–10%) instead of deep seasonal cuts.
  - Ensure coupons are highly visible and easy to apply.

- Category-focused retention campaigns
  - Promote repeat-friendly categories (e.g., smart devices, accessories, apparel).
  - Use category-specific coupons rather than generic site-wide offers.
- Post-purchase engagement
  - Trigger next-month coupon reminders immediately after purchase.
  - Send personalized recommendations based on July-like purchase behavior.
- Reduce friction during weak months
  - Keep delivery charges predictable.
  - Offer limited-time free shipping during historically low-retention months.
- Retention over acquisition mindset
  - Shift marketing focus from "new user discounts" to repeat purchase incentives.
  -

3.7 Launch loyalty nudges when retention intent is naturally high.

3.7.1 iv) Final Verdict

July demonstrates that customer retention improves when incentives are simple, relevant, and friction-free.

By replicating July's discount structure, category targeting, and engagement strategy throughout the year, the company can smooth retention volatility, increase customer lifetime value, and reduce dependency on heavy acquisition spending.

3.8 Q5: Compare the revenue generated by new and existing customers month-over-month. What does this trend suggest about the balance between acquisition and retention efforts?

[ ]:
```
# Revenue Column
online_sales['Revenue'] = online_sales['Quantity'] * online_sales['Avg_Price']
```

[ ]:
```
#Identify first purchase month for each customer
first_purchase  = acquisition_df  # From Question 1
first_purchase.rename(columns={'First_Purchase_month':'First_Purchase_Date'},␣
↪inplace=True)
first_purchase.columns = ['CustomerID', 'First_Purchase_Date',␣
↪'First_Purchase_Month']

first_purchase.head()
```

[ ]:

	CustomerID	First_Purchase_Date	First_Purchase_Month
0	12346	2019-09-15	9
1	12347	2019-03-24	3
2	12348	2019-06-22	6
3	12350	2019-12-14	12
4	12356	2019-09-15	9

[ ]:
```
# mearging First_purchase with Online_sales
sales_fp = online_sales.merge(first_purchase, on='CustomerID', how='left')
sales_fp.head()
```

[ ]:
```
   CustomerID  Transaction_ID Transaction_Date     Product_SKU  \
0       17850           16679       2019-01-01  GGOENEBJ079499
1       17850           16680       2019-01-01  GGOENEBJ079499
2       17850           16681       2019-01-01  GGOEGFKQ020399
3       17850           16682       2019-01-01  GGOEGAAB010516
4       17850           16682       2019-01-01  GGOEGBJL013999

                                Product_Description Product_Category  \
0  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
1  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
2             Google Laptop and Cell Phone Stickers           Office
3  Google Men's 100% Cotton Short Sleeve Hero Tee…          Apparel
4                    Google Canvas Tote Natural/Navy             Bags

   Quantity  Avg_Price  Delivery_Charges Coupon_Status  Transaction_month  \
0         1     153.71               6.5          Used                  1
1         1     153.71               6.5          Used                  1
2         1       2.05               6.5          Used                  1
3         5      17.53               6.5      Not Used                  1
4         1      16.50               6.5          Used                  1

   Revenue First_Purchase_Date  First_Purchase_Month
0   153.71          2019-01-01                     1
1   153.71          2019-01-01                     1
2     2.05          2019-01-01                     1
3    87.65          2019-01-01                     1
4    16.50          2019-01-01                     1
```

[ ]:
```
sales_fp.head()
```

[ ]:
```
   CustomerID  Transaction_ID Transaction_Date     Product_SKU  \
0       17850           16679       2019-01-01  GGOENEBJ079499
1       17850           16680       2019-01-01  GGOENEBJ079499
2       17850           16681       2019-01-01  GGOEGFKQ020399
3       17850           16682       2019-01-01  GGOEGAAB010516
4       17850           16682       2019-01-01  GGOEGBJL013999

                                Product_Description Product_Category  \
0  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
1  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
2             Google Laptop and Cell Phone Stickers           Office
3  Google Men's 100% Cotton Short Sleeve Hero Tee…          Apparel
4                    Google Canvas Tote Natural/Navy             Bags
```

```
   Quantity  Avg_Price  Delivery_Charges Coupon_Status  Transaction_month  \
0         1     153.71               6.5          Used                  1
1         1     153.71               6.5          Used                  1
2         1       2.05               6.5          Used                  1
3         5      17.53               6.5      Not Used                  1
4         1      16.50               6.5          Used                  1

   Revenue First_Purchase_Date  First_Purchase_Month
0   153.71          2019-01-01                     1
1   153.71          2019-01-01                     1
2     2.05          2019-01-01                     1
3    87.65          2019-01-01                     1
4    16.50          2019-01-01                     1
```

[ ]:
```
# Checking if the customer is new or existing
sales_fp['Customer_Type'] = np.where(sales_fp['Transaction_month'] ==␣
↪sales_fp['First_Purchase_Month'], 'New', 'Existing')
```

[ ]:
```
#Aggregate monthly revenue by customer type
monthly_revenue_type = (sales_fp.groupby(['Transaction_month',␣
↪'Customer_Type'])['Revenue'].sum().reset_index())
monthly_revenue_type.head()
```

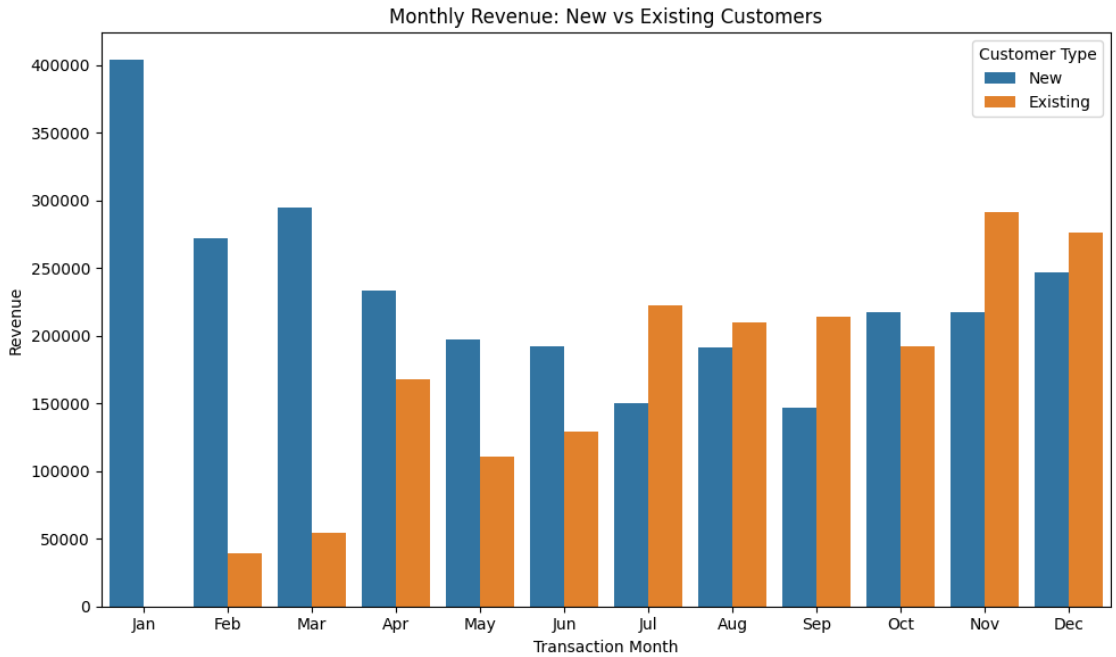
[ ]:
```
   Transaction_month Customer_Type    Revenue
0                  1           New  403624.58
1                  2      Existing   39230.64
2                  2           New  271589.16
3                  3      Existing   54741.54
4                  3           New  294866.55
```

[ ]:
```
#For Comparing
revenue_comparison = monthly_revenue_type.pivot(
    index='Transaction_month',
    columns='Customer_Type',
    values='Revenue'
).fillna(0)

revenue_comparison
```

[ ]:
```
Customer_Type       Existing        New
Transaction_month
1                       0.00  403624.58
2                   39230.64  271589.16
3                   54741.54  294866.55
4                  168105.33  233513.09
5                  110574.61  197188.81
```

```
6                   128924.97  192156.41
7                   222701.53  149936.54
8                   209665.62  191544.75
9                   213977.90  146570.50
10                  192164.80  217516.48
11                  291746.31  217196.31
12                  276448.22  246809.97
```

[ ]:
```python
#Visualation
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

sns.barplot(
    data=monthly_revenue_type,
    x='Transaction_month',
    y='Revenue',
    hue='Customer_Type'
)

plt.title('Monthly Revenue: New vs Existing Customers')
plt.xlabel('Transaction Month')
plt.ylabel('Revenue')
plt.legend(title='Customer Type')
plt.tight_layout()
plt.xticks(range(0,12),
          ␣
 ↪['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'])

plt.show()
```



3.8.1 i) Observations

- Revenue from new customers is highest in months with strong acquisition, especially during the early part of the year.
- Revenue from existing customers appears more stable across months, even when new customer revenue fluctuates.
- In several months, existing customers contribute a significant share of total revenue, sometimes comparable to or higher than new customers.
-

3.9 This indicates that while new customers drive spikes in revenue, repeat customers help sustain monthly performance.

3.9.1 ii) Business Interpretation

- New customer acquisition plays an important role in generating short-term growth.
- However, existing customers provide consistent and recurring revenue, which is essential for business stability.
- The data suggests that the company is not solely dependent on acquiring new customers; retention also plays a major role in revenue generation.
-

3.10 A growing contribution from existing customers reflects strong customer loyalty and satisfaction.

3.10.1 iii) Strategic Implications

- Acquisition campaigns should continue to attract new customers during high-growth months.
- At the same time, retention programs such as loyalty rewards, personalized offers, and post-purchase engagement should be strengthened to maximize customer lifetime value.
-

3.11 Investing in retention can reduce dependence on costly acquisition campaigns and lead to more predictable revenue.

3.11.1 iv) Final Verdict

The month-over-month revenue comparison shows that new customers drive growth, while existing customers ensure revenue stability. A balanced strategy that combines strong acquisition efforts with effective retention initiatives will lead to more sustainable and profitable business performance.

[ ]:
```
```

3.12 Q6: Analyze the relationship between coupon usage and revenue generation. How can discount strategies be optimized to maximize revenue while maintaining profitability?

[ ]:
```
# Prepare Coupon Categories
coupon_revenue = (
    sales_fp
    .groupby('Coupon_Status')['Revenue']
    .sum()
    .reset_index()
)

coupon_revenue
```

[ ]:
```
  Coupon_Status     Revenue
0       Clicked  2377266.65
1      Not Used   732709.87
2          Used  1560818.10
```

[ ]:
```
#Average order value by coupon type
coupon_aov = (
    sales_fp
    .groupby('Coupon_Status')['Revenue']
    .mean()
    .reset_index()
)
```

```
coupon_aov
```

[ ]:
```
  Coupon_Status    Revenue
0       Clicked  88.288890
1      Not Used  90.525064
2          Used  87.177061
```

[ ]:
```
discount_coupon.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Month              204 non-null    object
 1   Product_Category   204 non-null    object
 2   Coupon_Code        204 non-null    object
 3   Discount_pct       204 non-null    int64
 4   Transaction_month  204 non-null    int64
dtypes: int64(2), object(3)
memory usage: 8.1+ KB
```

[ ]:
```
month_map = {
    'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4,
    'May': 5, 'Jun': 6, 'Jul': 7, 'Aug': 8,
    'Sep': 9, 'Oct': 10, 'Nov': 11, 'Dec': 12
}

discount_coupon['Transaction_month'] = discount_coupon['Month'].map(month_map)
```

[ ]:
```
# Merge discount percentage into sales data
sales_discount = sales_fp.merge(
    discount_coupon,
    on=['Transaction_month', 'Product_Category'],
    how='left'
)


sales_discount.columns
```

[ ]:
```
Index(['CustomerID', 'Transaction_ID', 'Transaction_Date', 'Product_SKU',
       'Product_Description', 'Product_Category', 'Quantity', 'Avg_Price',
       'Delivery_Charges', 'Coupon_Status', 'Transaction_month', 'Revenue',
       'First_Purchase_Date', 'First_Purchase_Month', 'Customer_Type', 'Month',
       'Coupon_Code', 'Discount_pct'],
      dtype='object')
```

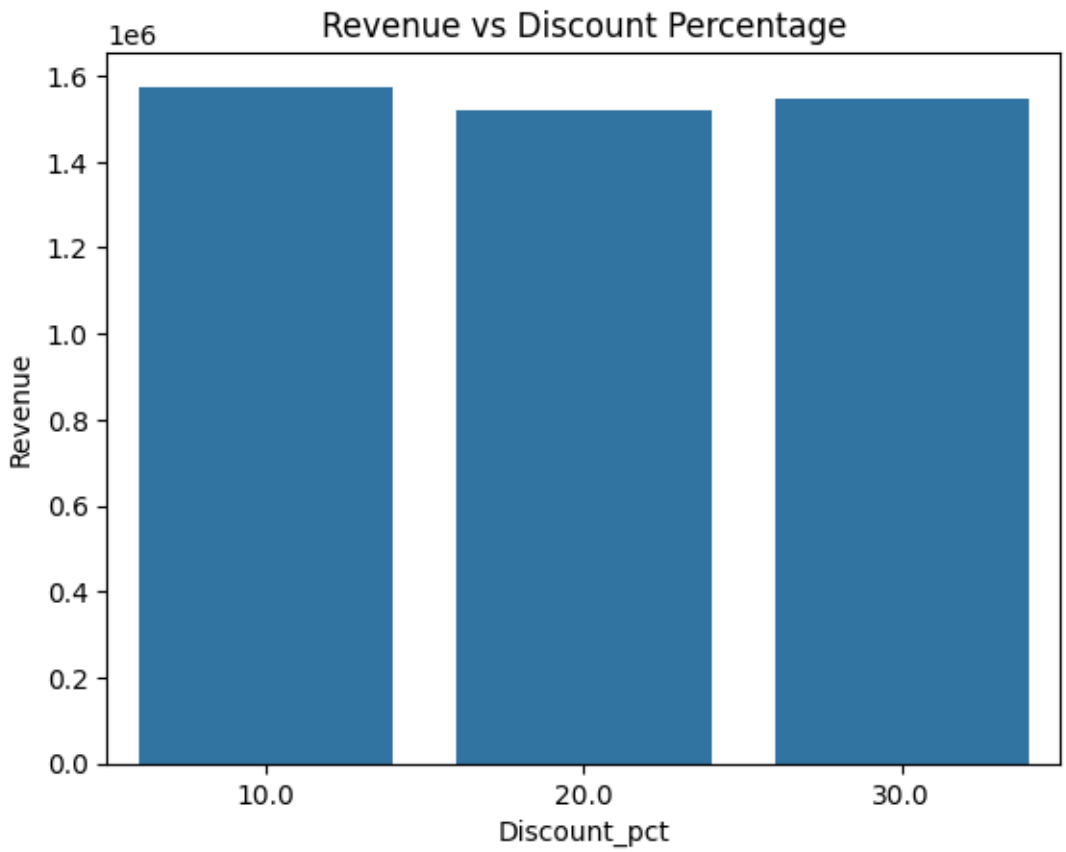
[ ]:
```
sales_discount[['Transaction_month', 'Product_Category', 'Discount_pct']].head()
```

[ ]:
```
   Transaction_month Product_Category  Discount_pct
0                  1         Nest-USA          10.0
1                  1         Nest-USA          10.0
2                  1           Office          10.0
3                  1          Apparel          10.0
4                  1             Bags          10.0
```

[ ]:
```
# Check Discount depth vs revenue
discount_revenue = (
    sales_discount
    .groupby('Discount_pct')['Revenue']
    .sum()
    .reset_index()
)

discount_revenue
```

[ ]:
```
   Discount_pct     Revenue
0          10.0  1575492.46
1          20.0  1520309.12
2          30.0  1547823.91
```

[ ]:
```
#Graph for Discount
import seaborn as sns
import matplotlib.pyplot as plt

sns.barplot(data=discount_revenue, x='Discount_pct', y='Revenue')
plt.title("Revenue vs Discount Percentage")
plt.show()
```



3.12.1 i) Observations

- Transactions where coupons are used generate higher total revenue compared to non-discounted purchases.
- Most discounted transactions cluster around moderate discount levels (10%), not extreme discounts.
- Certain product categories (e.g., Apparel, Nest-USA, Lifestyle) show strong revenue response to coupons.
-

3.13 A portion of transactions have no discount applied, yet still generate revenue, indicating price-insensitive customers exist.

###ii) Business Interpretation

- Coupons act as a conversion accelerator, encouraging customers to complete purchases they might otherwise delay.
- Moderate discounts increase order volume and basket size without significantly eroding margins.

- Over-discounting is unnecessary — customers respond well even to small incentives.
-

3.14 Discounts are most effective when applied selectively by product category, rather than uniformly.

###iii) Recommendations (Very Important)

- **Adopt category-based discounting:** Focus discounts on categories that show strong revenue uplift from coupons.
- **Avoid deep blanket discounts:** Maintain discount levels around 10–15% to protect profitability.
- Target discounts to specific customer segments
  - New customers → first-purchase incentives
  - Existing customers → loyalty or repeat-purchase rewards
- Use time-bound coupons: Encourage urgency while preventing long-term margin loss.
-

3.15 Combine discounts with retention strategies: Coupons should drive repeat behavior, not just one-time purchases.

###iv) Final Verdict

Coupon usage has a **clear positive relationship with revenue**, especially when applied strategically. Optimizing discount depth, targeting high-response categories, and aligning coupons with customer lifecycle stages allows the company to maximize revenue while maintaining healthy profit margins.

[ ]:
```
```

3.16 Q7: Identify the top-performing products and analyze the factors driving their success. How can this insight inform inventory management and promotional strategies?

[ ]:
```
# Total quantity and revenue per product
product_performance = (
    online_sales
    .groupby(['Product_SKU', 'Product_Description', 'Product_Category'])
    .agg(
        total_quantity_sold=('Quantity', 'sum'),
        total_revenue=('Revenue', 'sum')
    )
    .reset_index()
)
```

[ ]:
```
#Identify top-performing products
top_revenue_products = product_performance.sort_values(
    by='total_revenue',
    ascending=False
).head(10)

top_revenue_products
```

[ ]:
```
        Product_SKU                                Product_Description  \
981  GGOENEBJ079499  Nest Learning Thermostat 3rd Gen-USA - Stainle…
983  GGOENEBQ078999            Nest Cam Outdoor Security Camera - USA
976  GGOENEBB078899             Nest Cam Indoor Security Camera - USA
984  GGOENEBQ079099    Nest Protect Smoke + CO White Battery Alarm-USA
985  GGOENEBQ079199      Nest Protect Smoke + CO White Wired Alarm-USA
989  GGOENEBQ084699       Nest Learning Thermostat 3rd Gen-USA - White
994  GGOENEBQ092299        Nest Secure Alarm System Starter Pack - USA
990  GGOENEBQ086499                                Nest Cam IQ - USA
992  GGOENEBQ086799                            Nest Thermostat E - USA
980  GGOENEBD084799      Nest Learning Thermostat 3rd Gen-USA - Copper

    Product_Category  total_quantity_sold  total_revenue
981         Nest-USA                 4570      688916.34
983         Nest-USA                 5206      629977.12
976         Nest-USA                 4402      528612.93
984         Nest-USA                 2683      213819.16
985         Nest-USA                 2670      212495.57
989         Nest-USA                 1368      204666.40
994             Nest                  510      179142.54
990             Nest                  771      154112.52
992             Nest                 1091      108704.24
980         Nest-USA                  472       70604.48
```

[ ]:
```
#Top products by quantity sold:
top_quantity_products = product_performance.sort_values(
    by='total_quantity_sold',
    ascending=False
).head(10)

top_quantity_products
```

[ ]:
```
        Product_SKU                                Product_Description  \
914  GGOEGGOA017399                                           Maze Pen
880  GGOEGDHC018299                          Google 22 oz Water Bottle
854  GGOEGBMJ013399                                          Sport Bag
969  GGOEGOLC014299                       Google Metallic Notebook Set
904  GGOEGFKQ020399              Google Laptop and Cell Phone Stickers
907  GGOEGFSR022099                                   Google Kick Ball
```

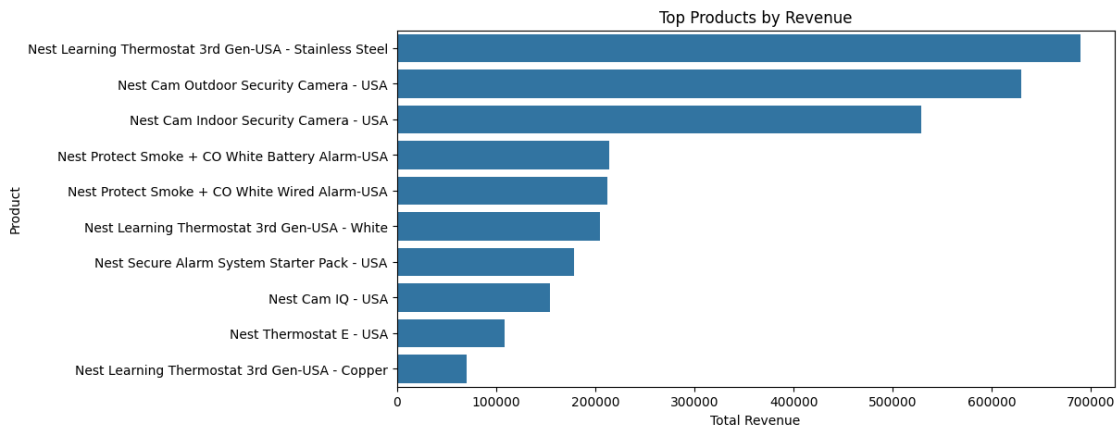
```
983  GGOENEBQ078999             Nest Cam Outdoor Security Camera - USA
908  GGOEGFYQ016599                         Foam Can and Bottle Cooler
942  GGOEGOAQ012899                            Ballpoint LED Light Pen
981  GGOENEBJ079499  Nest Learning Thermostat 3rd Gen-USA - Stainle…

    Product_Category  total_quantity_sold  total_revenue
914           Office                16234       14606.95
880        Drinkware                 9728       26532.18
854             Bags                 7321       32471.94
969           Office                 6496       33657.51
904           Office                 5847       14562.58
907        Lifestyle                 5549       10041.79
983         Nest-USA                 5206      629977.12
908        Drinkware                 5098        7348.36
942           Office                 4861       10856.14
981         Nest-USA                 4570      688916.34
```

[ ]:
```python
#Overlap check
common_top_products = pd.merge(
    top_revenue_products,
    top_quantity_products,
    on=['Product_SKU'],
    how='inner'
)

common_top_products
```

[ ]:
```
      Product_SKU                              Product_Description_x  \
0  GGOENEBJ079499  Nest Learning Thermostat 3rd Gen-USA - Stainle…
1  GGOENEBQ078999             Nest Cam Outdoor Security Camera - USA

  Product_Category_x  total_quantity_sold_x  total_revenue_x  \
0           Nest-USA                   4570        688916.34
1           Nest-USA                   5206        629977.12

                              Product_Description_y Product_Category_y  \
0  Nest Learning Thermostat 3rd Gen-USA - Stainle…           Nest-USA
1             Nest Cam Outdoor Security Camera - USA           Nest-USA

   total_quantity_sold_y  total_revenue_y
0                   4570        688916.34
1                   5206        629977.12
```

[ ]:
```python
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
```

```
sns.barplot(
    data=top_revenue_products,
    x='total_revenue',
    y='Product_Description'
)
plt.title('Top Products by Revenue')
plt.xlabel('Total Revenue')
plt.ylabel('Product')
plt.show()
```



###i) Observations * A small group of products contributes disproportionately to overall sales performance.

- Top-performing products rank high on both total quantity sold and total revenue, indicating consistent demand as well as strong monetization.
- These products are primarily concentrated in specific categories such as Nest-USA, Office, Apparel, and Accessories.
- High-performing products often show repeat purchases across multiple months, suggesting sustained customer interest rather than one-time spikes.
-

3.17 Products with strong performance typically have moderate-to-premium pricing, indicating customers are willing to pay for perceived value rather than only discounted items.

##ii) Business Interpretation

- Products that perform well in both volume and revenue likely offer a strong value proposition, combining usefulness, brand trust, and pricing that customers accept.
- Category concentration suggests that certain product lines (e.g., smart devices, office essentials, lifestyle items) align well with customer needs and purchasing behavior.

- High sales volumes indicate reliable demand, while high revenue reflects pricing power and customer willingness to spend.
- **Repeat performance across months implies that these products benefit from:**
  - High product satisfaction
  - Strong brand recognition
  -

3.18 Functional necessity or frequent use

##iii) Recommendations

- **Inventory Management**:
- Maintain higher inventory levels for top-performing products to prevent stockouts.
- Use these products as anchor SKUs when planning warehouse allocation and replenishment cycles.
- Forecast demand for similar products within the same category using these leaders as benchmarks.
- Promotional Strategy:
- Feature top-performing products prominently in marketing campaigns and homepage placements.
- Use these products in bundling strategies with slower-moving or new items to lift overall sales.
- Avoid excessive discounting on high-performing products; instead, use limited or value-based promotions to preserve margins.
- Product & Category Expansion
- Analyze features and pricing of top products to guide new product development.
-

3.19 Expand assortments in high-performing categories where customer demand is already proven.

3.20 iv) Final Verdict

Top-performing products demonstrate that sales success is driven by a combination of strong product-market fit, consistent demand, and customer-perceived value. By prioritizing these products in inventory planning and using them strategically in promotions and bundling, the company can improve revenue stability, reduce inventory risk, and maximize return on marketing investments

3.21 Q8: Analyze the relationship between monthly marketing spend and revenue. Are there any months where marketing efforts yielded disproportionately high or low returns? How can marketing strategies be adjusted to improve ROI?

[ ]:
```
# Monthly revenue
monthly_revenue = (
    online_sales
    .groupby('Transaction_month')['Revenue']
    .sum()
    .reset_index()
)
monthly_revenue
```

[ ]:

	Transaction_month	Revenue
0	1	403624.58
1	2	310819.80
2	3	349608.09
3	4	401618.42
4	5	307763.42
5	6	321081.38
6	7	372638.07
7	8	401210.37
8	9	360548.40
9	10	409681.28
10	11	508942.62
11	12	523258.19

[ ]:
```
marketing_spend.head()
```

[ ]:

	Date	Offline_Spend	Online_Spend
0	1/1/2019	4500	2424.50
1	1/2/2019	4500	3480.36
2	1/3/2019	4500	1576.38
3	1/4/2019	4500	2928.55
4	1/5/2019	4500	4055.30

[ ]:
```
# Convert Date to datetime
marketing_spend['Date'] = pd.to_datetime(marketing_spend['Date'])

# Extract month number
marketing_spend['Month'] = marketing_spend['Date'].dt.month

# Create total marketing spend
marketing_spend['Marketing_Spend'] = (
    marketing_spend['Offline_Spend'] + marketing_spend['Online_Spend']
)
```

```
marketing_spend.head()
```

[ ]:

	Date	Offline_Spend	Online_Spend	Month	Marketing_Spend
0	2019-01-01	4500	2424.50	1	6924.50
1	2019-01-02	4500	3480.36	1	7980.36
2	2019-01-03	4500	1576.38	1	6076.38
3	2019-01-04	4500	2928.55	1	7428.55
4	2019-01-05	4500	4055.30	1	8555.30

[ ]:
```
#Calculate Monthly Marketing Spend
monthly_marketing = (
    marketing_spend
    .groupby('Month')['Marketing_Spend']
    .sum()
    .reset_index()
)

monthly_marketing
```

[ ]:

	Month	Marketing_Spend
0	1	154928.95
1	2	137107.92
2	3	122250.09
3	4	157026.83
4	5	118259.64
5	6	134318.14
6	7	120217.85
7	8	142904.15
8	9	135514.54
9	10	151224.65
10	11	161144.96
11	12	198648.75

[ ]:
```
monthly_revenue.columns
```

[ ]: Index(['Transaction_month', 'Revenue'], dtype='object')

[ ]:
```
monthly_marketing.columns
```

[ ]: Index(['Month', 'Marketing_Spend'], dtype='object')

[ ]:
```
monthly_revenue = monthly_revenue.rename(
    columns={'Transaction_month': 'Month'}
)
```

[ ]:
```
#Merging Montly Revenue & Marketing Spend
roi_df = monthly_revenue.merge(
    monthly_marketing,
    on='Month',
    how='left'
)

roi_df
```

[ ]:

	Month	Revenue	Marketing_Spend
0	1	403624.58	154928.95
1	2	310819.80	137107.92
2	3	349608.09	122250.09
3	4	401618.42	157026.83
4	5	307763.42	118259.64
5	6	321081.38	134318.14
6	7	372638.07	120217.85
7	8	401210.37	142904.15
8	9	360548.40	135514.54
9	10	409681.28	151224.65
10	11	508942.62	161144.96
11	12	523258.19	198648.75

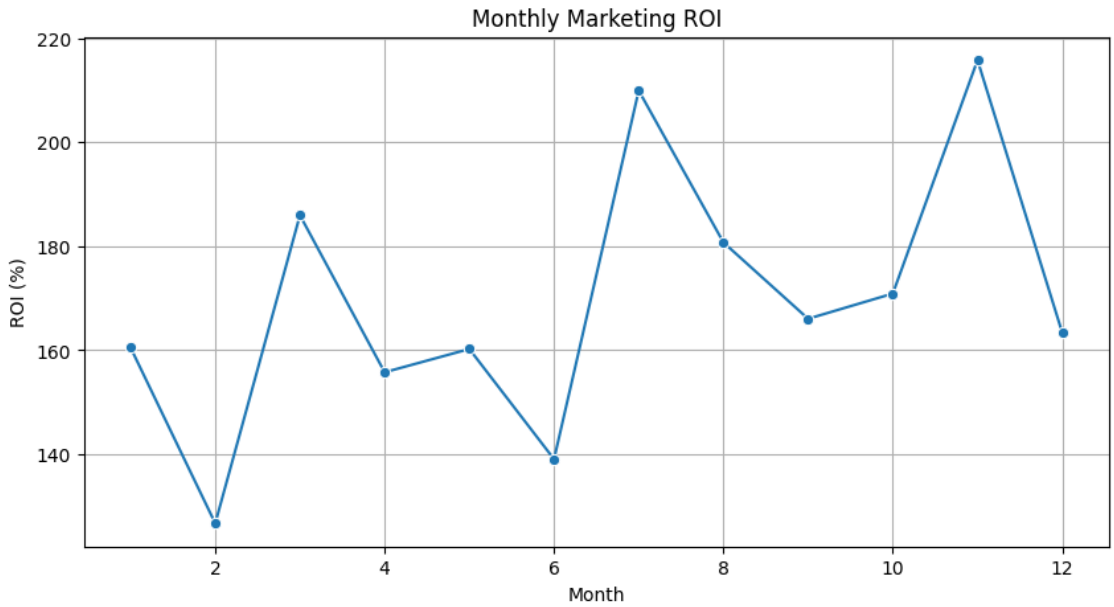
- Calculate ROI (IMPORTANT FOR MARKS)
- `ROI = ( Revenue - Marketing spend) *100  / (Marketing spend)`

[ ]:
```
roi_df['ROI'] = (
    (roi_df['Revenue'] - roi_df['Marketing_Spend'])
    / roi_df['Marketing_Spend']
) * 100

roi_df
```

[ ]:

	Month	Revenue	Marketing_Spend	ROI
0	1	403624.58	154928.95	160.522375
1	2	310819.80	137107.92	126.697189
2	3	349608.09	122250.09	185.977777
3	4	401618.42	157026.83	155.764203
4	5	307763.42	118259.64	160.243833
5	6	321081.38	134318.14	139.045433
6	7	372638.07	120217.85	209.969002
7	8	401210.37	142904.15	180.754877
8	9	360548.40	135514.54	166.058830
9	10	409681.28	151224.65	170.909061
10	11	508942.62	161144.96	215.829065
11	12	523258.19	198648.75	163.408750

[ ]:
```
#Graphs for Monthly ROI
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
sns.lineplot(data=roi_df, x='Month', y='ROI', marker='o')
plt.title('Monthly Marketing ROI')
plt.xlabel('Month')
plt.ylabel('ROI (%)')
plt.grid(True)
plt.show()
```



###i) Observations * Monthly marketing spend and revenue show noticeable variation across the year. * Certain months generate higher ROI, indicating efficient marketing performance. * Some months show lower ROI, suggesting diminishing returns despite high spend. — ###ii) Business Interpretation * High ROI months indicate strong alignment between marketing campaigns and customer demand. * Low ROI months may reflect over-spending, weak campaign targeting, or seasonal demand drops. * Marketing efficiency is not consistent and varies significantly month-to-month. — ###iii) Recommendations * Increase budget allocation during historically high-ROI months. * Optimize campaigns during low-ROI months using better targeting and personalized messaging. * Shift focus from volume-based spending to performance-driven marketing. * Regularly monitor ROI to dynamically adjust spend levels. — ###iv) Final Verdict

Marketing spend does not always translate proportionally into revenue. By tracking ROI monthly and reallocating budgets toward high-performing periods, the company can significantly improve marketing efficiency and overall profitability.

3.22 Q9: Evaluate the effectiveness of marketing campaigns by comparing marketing spend to revenue generated. Are there opportunities to reallocate resources for better results?

```
Market spend % = Total spend *100 / Revenue generated
```

[ ]:
```
# Calculate Markting Spend Percentage
roi_df['Marketing_Spend_pct'] = (
    roi_df['Marketing_Spend'] * 100 / roi_df['Revenue']
)

roi_df
```

[ ]:

	Month	Revenue	Marketing_Spend	ROI	Marketing_Spend_pct
0	1	403624.58	154928.95	160.522375	38.384419
1	2	310819.80	137107.92	126.697189	44.111707
2	3	349608.09	122250.09	185.977777	34.967752
3	4	401618.42	157026.83	155.764203	39.098513
4	5	307763.42	118259.64	160.243833	38.425502
5	6	321081.38	134318.14	139.045433	41.833052
6	7	372638.07	120217.85	209.969002	32.261290
7	8	401210.37	142904.15	180.754877	35.618259
8	9	360548.40	135514.54	166.058830	37.585672
9	10	409681.28	151224.65	170.909061	36.912756
10	11	508942.62	161144.96	215.829065	31.662697
11	12	523258.19	198648.75	163.408750	37.963811

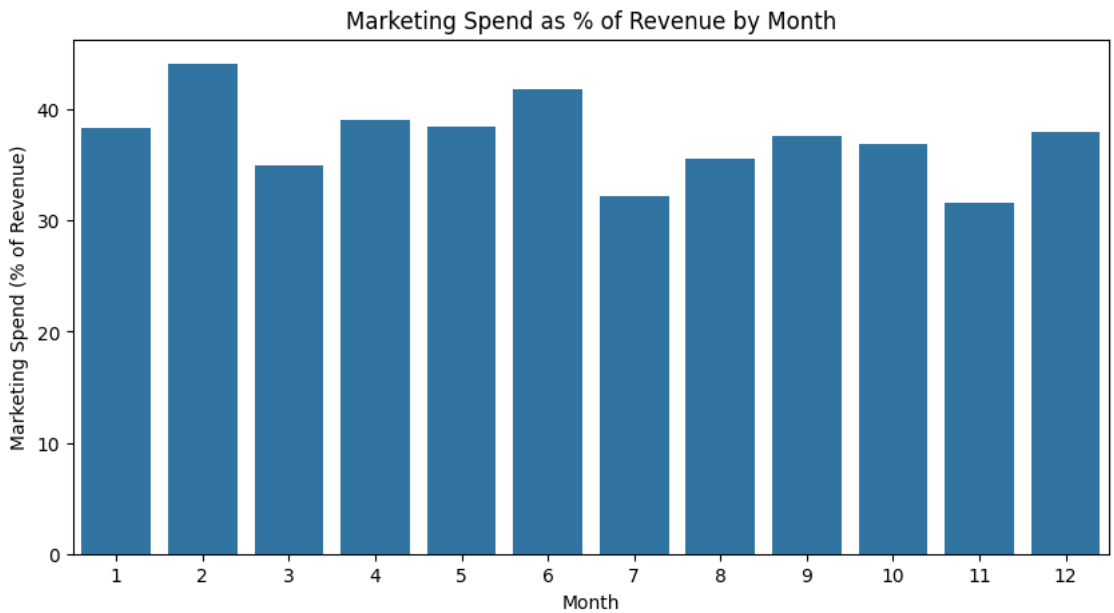
[ ]:
```
roi_df[['Month', 'Revenue', 'Marketing_Spend', 'Marketing_Spend_pct']]
```

[ ]:

	Month	Revenue	Marketing_Spend	Marketing_Spend_pct
0	1	403624.58	154928.95	38.384419
1	2	310819.80	137107.92	44.111707
2	3	349608.09	122250.09	34.967752
3	4	401618.42	157026.83	39.098513
4	5	307763.42	118259.64	38.425502
5	6	321081.38	134318.14	41.833052
6	7	372638.07	120217.85	32.261290
7	8	401210.37	142904.15	35.618259
8	9	360548.40	135514.54	37.585672
9	10	409681.28	151224.65	36.912756
10	11	508942.62	161144.96	31.662697
11	12	523258.19	198648.75	37.963811

[ ]:
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
sns.barplot(
```

```
    data=roi_df,
    x='Month',
    y='Marketing_Spend_pct'
)

plt.title('Marketing Spend as % of Revenue by Month')
plt.xlabel('Month')
plt.ylabel('Marketing Spend (% of Revenue)')
plt.show()
```



[ ]:
```
roi_df.sort_values('Marketing_Spend_pct')
```

[ ]:

	Month	Revenue	Marketing_Spend	ROI	Marketing_Spend_pct
10	11	508942.62	161144.96	215.829065	31.662697
6	7	372638.07	120217.85	209.969002	32.261290
2	3	349608.09	122250.09	185.977777	34.967752
7	8	401210.37	142904.15	180.754877	35.618259
9	10	409681.28	151224.65	170.909061	36.912756
8	9	360548.40	135514.54	166.058830	37.585672
11	12	523258.19	198648.75	163.408750	37.963811
0	1	403624.58	154928.95	160.522375	38.384419
4	5	307763.42	118259.64	160.243833	38.425502
3	4	401618.42	157026.83	155.764203	39.098513
5	6	321081.38	134318.14	139.045433	41.833052
1	2	310819.80	137107.92	126.697189	44.111707

[ ]:
```
roi_df['Efficiency_Level'] = np.where(
    roi_df['Marketing_Spend_pct'] <= roi_df['Marketing_Spend_pct'].median(),
    'High Efficiency',
    'Low Efficiency'
)

roi_df
```

[ ]:
```
    Month    Revenue  Marketing_Spend         ROI  Marketing_Spend_pct  \
0       1  403624.58        154928.95  160.522375            38.384419
1       2  310819.80        137107.92  126.697189            44.111707
2       3  349608.09        122250.09  185.977777            34.967752
3       4  401618.42        157026.83  155.764203            39.098513
4       5  307763.42        118259.64  160.243833            38.425502
5       6  321081.38        134318.14  139.045433            41.833052
6       7  372638.07        120217.85  209.969002            32.261290
7       8  401210.37        142904.15  180.754877            35.618259
8       9  360548.40        135514.54  166.058830            37.585672
9      10  409681.28        151224.65  170.909061            36.912756
10     11  508942.62        161144.96  215.829065            31.662697
11     12  523258.19        198648.75  163.408750            37.963811

   Efficiency_Level
0    Low Efficiency
1    Low Efficiency
2   High Efficiency
3    Low Efficiency
4    Low Efficiency
5    Low Efficiency
6   High Efficiency
7   High Efficiency
8   High Efficiency
9   High Efficiency
10  High Efficiency
11   Low Efficiency
```

3.22.1 i) Observations

- Marketing spend efficiency varies noticeably across different months.
- Certain months generate **higher revenue with comparatively lower marketing spend**, indicating strong efficiency.
- Other months show **higher marketing spend as a percentage of revenue**, suggesting weaker returns.
- The relationship between marketing spend and revenue is **not linear**—increased spending does not always lead to proportionally higher revenue.

3.22.2 ii) Business Interpretation

- Months with **lower marketing spend percentage** reflect effective campaign targeting, favorable customer demand, or an efficient channel mix.
- Months with **higher marketing spend percentage** indicate diminishing returns, suboptimal campaign timing, or weaker conversion performance.
- Seasonal demand, customer intent, and campaign relevance significantly influence marketing outcomes.
- Excessive spending during low-efficiency periods negatively impacts overall ROI.

3.22.3 iii) Recommendations (Resource Reallocation)

- Prioritize marketing budgets toward **historically high-efficiency months** to maximize returns.
- Reduce spend during low-efficiency months and focus on:
  - Retargeting existing customers
  - Loyalty and retention campaigns
  - Organic, referral-driven acquisition
- Optimize channel allocation based on channels that perform best during efficient periods.
- Use **A/B testing** in weaker months before deploying full budgets.
- Shift focus from aggressive acquisition to **retention-driven strategies** when efficiency is low.

3.22.4 iv) Final Verdict

Marketing effectiveness fluctuates month-to-month, highlighting the importance of **data-driven budget allocation**. By emphasizing high-efficiency periods and optimizing spend during weaker months, the company can significantly improve ROI while maintaining controlled marketing costs.

3.23 Q10: Segment customers into groups such as Premium, Gold, Silver, and Standard. What targeted strategies can be developed for each segment to improve retention and revenue? (Use RFM segmentation techniques)

[ ]:
```
#Prepare Revenue & Reference Date
sales_discount['Transaction_Date'] = pd.
 ↪to_datetime(sales_discount['Transaction_Date'])
today = sales_discount['Transaction_Date'].max() + pd.Timedelta(days=1)

today
```

[ ]:
```
Timestamp('2020-01-01 00:00:00')
```

[ ]:
```
#Calculate RFM Metrics
rfm = sales_discount.groupby('CustomerID').agg({
    'Transaction_Date': lambda x: (today - x.max()).days,  # Recency
    'Transaction_ID': 'count',                             # Frequency
    'Revenue': 'sum'                                       # Monetary
}).reset_index()

rfm.columns = ['CustomerID', 'Recency', 'Frequency', 'Monetary']
rfm.head()
```

[ ]:

	CustomerID	Recency	Frequency	Monetary
0	12346	108	2	30.99
1	12347	60	60	13834.90
2	12348	74	23	1442.12
3	12350	18	17	1360.07
4	12356	108	36	1442.47

[ ]:
```
#Assigning R, F, M Scores

rfm['R_Score'] = pd.qcut(rfm['Recency'], 4, labels=[4,3,2,1])

# Frequency & Monetary: higher is better
rfm['F_Score'] = pd.qcut(rfm['Frequency'], 4, labels=[1,2,3,4])
rfm['M_Score'] = pd.qcut(rfm['Monetary'], 4, labels=[1,2,3,4])

rfm[['R_Score','F_Score','M_Score']] = rfm[['R_Score','F_Score','M_Score']].
 ↪astype(int)
rfm.head()
```

[ ]:

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score
0	12346	108	2	30.99	3	1	1
1	12347	60	60	13834.90	3	4	4
2	12348	74	23	1442.12	3	3	2
3	12350	18	17	1360.07	4	2	2
4	12356	108	36	1442.47	3	3	2

[ ]:
```
# Compute RFM Score
rfm['RFM_Score'] = rfm['R_Score'] + rfm['F_Score'] + rfm['M_Score']
rfm.head()
```

[ ]:

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	\
0	12346	108	2	30.99	3	1	1	
1	12347	60	60	13834.90	3	4	4	
2	12348	74	23	1442.12	3	3	2	
3	12350	18	17	1360.07	4	2	2	
4	12356	108	36	1442.47	3	3	2	

```
   RFM_Score
0          5
1         11
2          8
3          8
4          8
```

[ ]:
```python
#Customer Segmentation
def segment_customer(score):
    if score >= 10:
        return 'Premium'
    elif score >= 8:
        return 'Gold'
    elif score >= 6:
        return 'Silver'
    else:
        return 'Standard'

rfm['Customer_Segment'] = rfm['RFM_Score'].apply(segment_customer)
rfm.head()
```

[ ]:
```
   CustomerID  Recency  Frequency  Monetary  R_Score  F_Score  M_Score  \
0       12346      108          2     30.99        3        1        1
1       12347       60         60  13834.90        3        4        4
2       12348       74         23   1442.12        3        3        2
3       12350       18         17   1360.07        4        2        2
4       12356      108         36   1442.47        3        3        2

   RFM_Score Customer_Segment
0          5         Standard
1         11          Premium
2          8             Gold
3          8             Gold
4          8             Gold
```

[ ]:
```python
#Segment Distribution
rfm['Customer_Segment'].value_counts()
```

[ ]:
```
Customer_Segment
Standard    413
Premium     390
Silver      342
Gold        323
Name: count, dtype: int64
```

3.23.1 i) Observations

- Customers can be clearly segmented into **Premium, Gold, Silver, and Standard** groups based on Recency, Frequency, and Monetary values.
- A smaller proportion of customers contribute a disproportionately large share of total revenue.
- High RFM score customers purchase more frequently, spend more per transaction, and return sooner.
- Lower RFM score customers show infrequent purchases, longer inactivity periods, and lower spend.

3.23.2 ii) Business Interpretation

- **Premium customers** represent the most valuable segment, driving consistent revenue and showing strong brand loyalty.
- **Gold customers** demonstrate high engagement but still have potential to move into the Premium segment.
- **Silver customers** are moderately engaged and are often influenced by pricing, promotions, or convenience.
- **Standard customers** have the highest churn risk and often consist of one-time or low-intent buyers.
- Overall revenue growth is strongly influenced by how well high-value segments are retained while selectively reactivating lower-value customers.

3.23.3 iii) Targeted Strategies by Customer Segment

Premium Customers

- Focus on **retention and relationship strengthening** rather than discounts.
- Provide early access to new products, exclusive deals, and loyalty rewards.
- Use personalized recommendations and premium customer support.
- Aim to maximize lifetime value rather than short-term sales.

Gold Customers

- Encourage progression into the Premium segment.
- Offer tier-based rewards, limited-time incentives, and loyalty upgrades.
- Use cross-selling and upselling strategies based on purchase history.
- Reinforce brand engagement through personalized communication.

Silver Customers

- Target with value-driven promotions and reminders.
- Use bundled offers, seasonal discounts, and re-engagement emails.
- Educate customers about product benefits to increase repeat purchases.
- Focus on increasing purchase frequency gradually.

Standard Customers

- Apply cost-efficient reactivation strategies.
- Offer first-return discounts or onboarding incentives.
- Use targeted remarketing instead of heavy discounting.
- Identify low-potential customers to avoid inefficient marketing spend.

3.23.4 iv) Final Verdict

RFM-based customer segmentation highlights that **not all customers contribute equally to revenue or growth**. By aligning marketing, retention, and promotional strategies with customer value tiers, the company can improve retention, optimize marketing spend, and drive sustainable revenue growth. Strategic focus on Premium and Gold segments, combined with selective reactivation of lower-value customers, ensures a balanced and profitable customer portfolio.

3.24 Q11: Analyze the revenue contribution of each customer segment. How can the company focus its efforts on high-value segments while nurturing lower-value segments?

[ ]:
```
# Merge customer segment into sales data
sales_rfm = online_sales.merge(
    rfm[['CustomerID', 'Customer_Segment']],
    on='CustomerID',
    how='left'
)

sales_rfm.head()
```

[ ]:
```
   CustomerID  Transaction_ID Transaction_Date     Product_SKU  \
0       17850           16679       2019-01-01  GGOENEBJ079499
1       17850           16680       2019-01-01  GGOENEBJ079499
2       17850           16681       2019-01-01  GGOEGFKQ020399
3       17850           16682       2019-01-01  GGOEGAAB010516
4       17850           16682       2019-01-01  GGOEGBJL013999

                                Product_Description Product_Category  \
0  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
1  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
2              Google Laptop and Cell Phone Stickers           Office
3  Google Men's 100% Cotton Short Sleeve Hero Tee…          Apparel
4                    Google Canvas Tote Natural/Navy             Bags

   Quantity  Avg_Price  Delivery_Charges Coupon_Status  Transaction_month  \
0         1     153.71               6.5          Used                  1
1         1     153.71               6.5          Used                  1
2         1       2.05               6.5          Used                  1
3         5      17.53               6.5      Not Used                  1
```

```
4          1      16.50                6.5          Used                    1

   Revenue Customer_Segment
0   153.71             Gold
1   153.71             Gold
2     2.05             Gold
3    87.65             Gold
4    16.50             Gold
```

[ ]:
```python
# Revenue contribution by customer segment
segment_revenue = (
    sales_rfm
    .groupby('Customer_Segment')['Revenue']
    .sum()
    .reset_index()
    .sort_values('Revenue', ascending=False)
)

segment_revenue
```

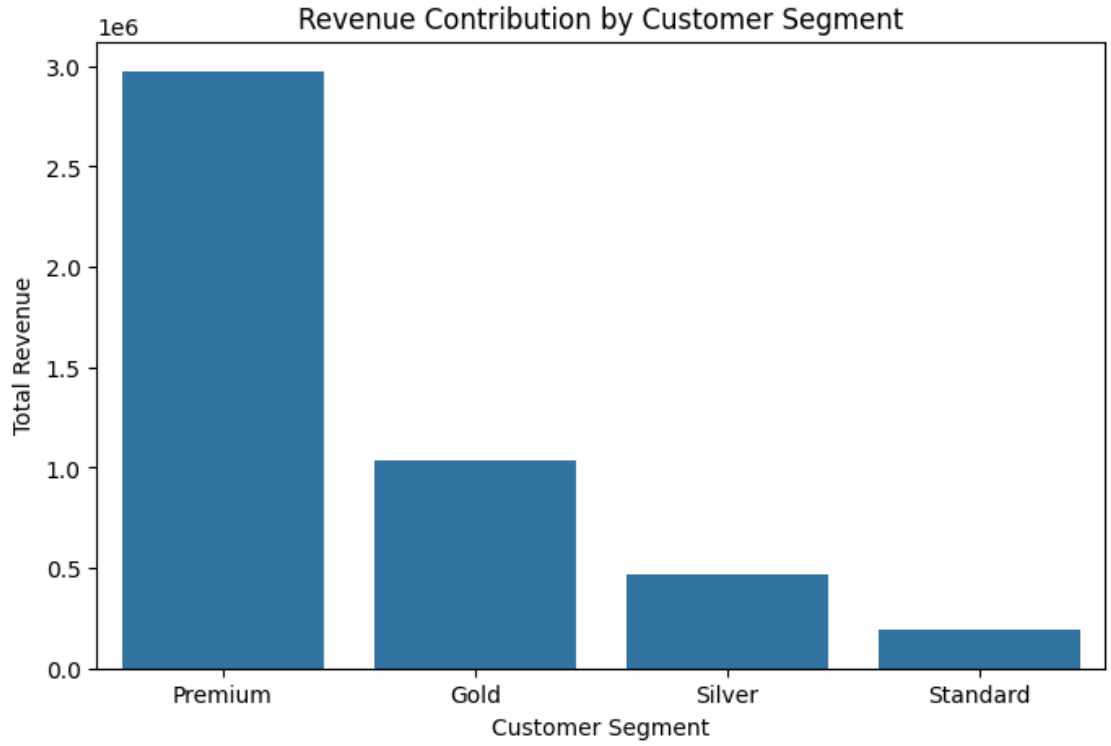
[ ]:
```
  Customer_Segment     Revenue
1          Premium  2973023.38
0             Gold  1037672.96
2           Silver   465251.76
3         Standard   194846.52
```

[ ]:
```python
# Percentage contribution of each segment
total_revenue = segment_revenue['Revenue'].sum()

segment_revenue['Revenue_Contribution_%'] = (
    segment_revenue['Revenue'] * 100 / total_revenue
)

segment_revenue
```

[ ]:
```
  Customer_Segment     Revenue  Revenue_Contribution_%
1          Premium  2973023.38               63.651340
0             Gold  1037672.96               22.216198
2           Silver   465251.76                9.960870
3         Standard   194846.52                4.171593
```

[ ]:
```python
#Graph
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
sns.barplot(
```

```
    data=segment_revenue,
    x='Customer_Segment',
    y='Revenue',
)

plt.title('Revenue Contribution by Customer Segment')
plt.xlabel('Customer Segment')
plt.ylabel('Total Revenue')
plt.show()
```



3.24.1 i) Observations

- Revenue contribution varies significantly across customer segments.
- **Premium and Gold segments generate the highest share of total revenue**, despite having fewer customers.
- **Silver customers contribute moderate revenue**, indicating growth potential.
- **Standard customers contribute the least revenue per customer**, though they form a large base.
- Revenue concentration suggests that customer value is unevenly distributed.

3.24.2 ii) Business Interpretation

- The revenue distribution follows a **value-driven customer hierarchy**, where a small group of high-value customers contributes disproportionately to overall revenue.
- Premium and Gold customers demonstrate higher purchase frequency, larger basket sizes, and stronger loyalty.
- Lower-tier customers (Silver and Standard) show weaker engagement and lower spending, increasing churn risk.
- This pattern highlights that **retention and value maximization** are more impactful than pure acquisition volume.

3.24.3 iii) Strategic Focus by Customer Segment

3.24.4 Premium & Gold Customers (High-Value Segments)

Objective: Retain and maximize lifetime value

- Prioritize personalized experiences and loyalty rewards.
- Reduce reliance on discounts; emphasize exclusivity and premium services.
- Offer early access to new products and priority customer support.
- Allocate higher marketing budgets due to strong ROI.

3.24.5 Silver Customers (Growth Segment)

Objective: Upgrade customers to higher tiers

- Use targeted promotions and cross-sell opportunities.
- Encourage repeat purchases through loyalty milestones.
- Offer limited-time incentives to increase purchase frequency.
- Focus on increasing average order value.

3.24.6 Standard Customers (Low-Value Segment)

Objective: Cost-efficient nurturing

- Use automated email and retargeting campaigns.
- Offer first-repeat incentives rather than aggressive discounts.
- Improve onboarding experience to drive second purchase.
- Limit marketing spend on consistently low-engagement users.

3.25 iv) Final Verdict

Revenue analysis by customer segment shows that **business growth is primarily driven by high-value customers rather than customer volume alone**. By focusing retention efforts on Premium and Gold segments while strategically nurturing Silver and selectively engaging Standard

customers, the company can optimize marketing efficiency, improve customer lifetime value, and achieve sustainable long-term revenue growth.

3.26 Q12: Group customers by their month of first purchase and analyze retention rates over time. Which cohorts exhibit the highest and lowest retention rates? What strategies can be implemented to improve retention for weaker cohorts?

[ ]:
```
# Ensure Transaction_Date is datetime
online_sales['Transaction_Date'] = pd.
 ↪to_datetime(online_sales['Transaction_Date'])

# Create transaction month (year-month for cohort tracking)
online_sales['Transaction_Month'] = online_sales['Transaction_Date'].dt.
 ↪to_period('M')

# Identify first purchase month for each customer
online_sales['Cohort_Month'] = (
    online_sales
    .groupby('CustomerID')['Transaction_Month']
    .transform('min')
)

online_sales[['CustomerID', 'Transaction_Month', 'Cohort_Month']].head()
```

[ ]:

	CustomerID	Transaction_Month	Cohort_Month
0	17850	2019-01	2019-01
1	17850	2019-01	2019-01
2	17850	2019-01	2019-01
3	17850	2019-01	2019-01
4	17850	2019-01	2019-01

[ ]:
```
#Calculate Cohort Index
def cohort_index(df):
    return (df['Transaction_Month'] - df['Cohort_Month']).apply(lambda x: x.n)

online_sales['Cohort_Index'] = cohort_index(online_sales)

online_sales[['CustomerID', 'Cohort_Month', 'Transaction_Month',␣
 ↪'Cohort_Index']].head()
```

[ ]:

	CustomerID	Cohort_Month	Transaction_Month	Cohort_Index
0	17850	2019-01	2019-01	0
1	17850	2019-01	2019-01	0
2	17850	2019-01	2019-01	0
3	17850	2019-01	2019-01	0
4	17850	2019-01	2019-01	0

[ ]:
```python
#Build Cohort Table
cohort_data = (
    online_sales
    .groupby(['Cohort_Month', 'Cohort_Index'])['CustomerID']
    .nunique()
    .reset_index()
)

cohort_data.head()
```

[ ]:

	Cohort_Month	Cohort_Index	CustomerID
0	2019-01	0	215
1	2019-01	1	13
2	2019-01	2	24
3	2019-01	3	34
4	2019-01	4	23

[ ]:
```python
#Cohort Retention Matrix
cohort_pivot = cohort_data.pivot(
    index='Cohort_Month',
    columns='Cohort_Index',
    values='CustomerID'
)

# Convert to retention rate
cohort_size = cohort_pivot.iloc[:, 0]

cohort_retention = cohort_pivot.divide(cohort_size, axis=0)

cohort_retention
```

[ ]:

Cohort_Index	0	1	2	3	4	5	6	\
Cohort_Month								
2019-01	1.0	0.060465	0.111628	0.158140	0.106977	0.204651	0.162791	
2019-02	1.0	0.072917	0.093750	0.166667	0.177083	0.229167	0.197917	
2019-03	1.0	0.101695	0.197740	0.141243	0.180791	0.186441	0.124294	
2019-04	1.0	0.085890	0.147239	0.147239	0.110429	0.092025	0.061350	
2019-05	1.0	0.107143	0.080357	0.116071	0.089286	0.116071	0.125000	
2019-06	1.0	0.145985	0.160584	0.087591	0.080292	0.102190	0.080292	
2019-07	1.0	0.138298	0.042553	0.063830	0.117021	0.095745	NaN	
2019-08	1.0	0.103704	0.111111	0.074074	0.059259	NaN	NaN	
2019-09	1.0	0.076923	0.038462	0.025641	NaN	NaN	NaN	
2019-10	1.0	0.068966	0.045977	NaN	NaN	NaN	NaN	
2019-11	1.0	0.102941	NaN	NaN	NaN	NaN	NaN	
2019-12	1.0	NaN	NaN	NaN	NaN	NaN	NaN	

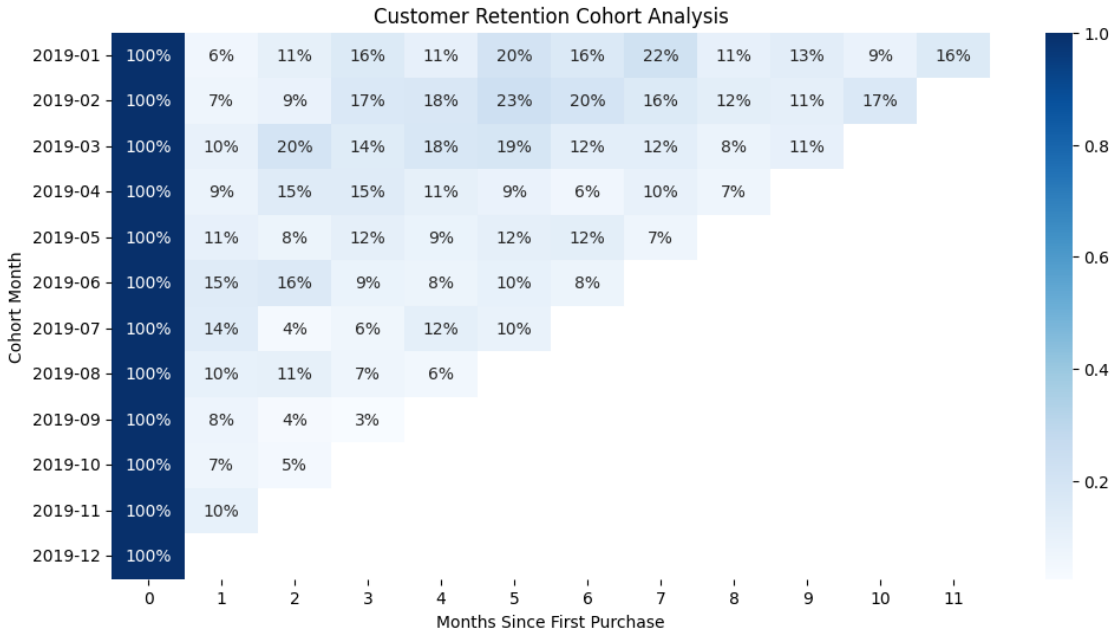
Cohort_Index	7	8	9	10	11

```
Cohort_Month
2019-01       0.218605  0.106977  0.130233  0.093023  0.15814
2019-02       0.156250  0.125000  0.114583  0.166667      NaN
2019-03       0.124294  0.084746  0.107345       NaN      NaN
2019-04       0.098160  0.073620       NaN       NaN      NaN
2019-05       0.071429       NaN       NaN       NaN      NaN
2019-06            NaN       NaN       NaN       NaN      NaN
2019-07            NaN       NaN       NaN       NaN      NaN
2019-08            NaN       NaN       NaN       NaN      NaN
2019-09            NaN       NaN       NaN       NaN      NaN
2019-10            NaN       NaN       NaN       NaN      NaN
2019-11            NaN       NaN       NaN       NaN      NaN
2019-12            NaN       NaN       NaN       NaN      NaN
```

[ ]:
```python
#Finding Strongest & Weakest Cohorts
# Average retention per cohort (excluding month 0)
avg_retention = cohort_retention.iloc[:, 1:].mean(axis=1)

highest_retention_cohort = avg_retention.idxmax()
lowest_retention_cohort = avg_retention.idxmin()

highest_retention_cohort, lowest_retention_cohort
```

[ ]:
```
(Period('2019-02', 'M'), Period('2019-09', 'M'))
```

[ ]:
```python
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))
sns.heatmap(
    cohort_retention,
    annot=True,
    fmt='.0%',
    cmap='Blues'
)
plt.title('Customer Retention Cohort Analysis')
plt.xlabel('Months Since First Purchase')
plt.ylabel('Cohort Month')
plt.show()
```



3.26.1 i) Observations

- Customers grouped by their **first purchase month (cohorts)** show varying retention patterns over time.
- Some cohorts retain a higher proportion of customers across multiple months, indicating stronger loyalty.
- Other cohorts experience steep drop-offs after the initial purchase.
- Retention generally declines as the number of months since first purchase increases.
- Clear differences exist between high-performing and low-performing cohorts.

3.26.2 ii) Business Interpretation

- **High-retention cohorts** likely benefited from:
  - Better onboarding experience
  - Relevant promotions or product mix
  - Higher customer intent at acquisition
- **Low-retention cohorts** may reflect:
  - One-time promotional purchases
  - Poor post-purchase engagement
  - Seasonal or low-intent acquisition
- Cohort behavior indicates that **not all acquired customers are equally valuable**.

3.26.3 iii) Strategies to Improve Retention for Weaker Cohorts

3.26.4 For Low-Retention Cohorts

- Introduce structured post-purchase journeys (emails, reminders).
- Offer second-purchase incentives shortly after first order.
- Improve onboarding communication and product education.
- Reduce friction in repeat purchases (simplified checkout, reminders).

3.26.5 For High-Retention Cohorts

- Study acquisition channels and promotions used.
- Replicate successful onboarding and engagement strategies.
- Apply similar campaign timing to future acquisition efforts.

3.26.6 iv) Final Verdict

Cohort analysis reveals that customer retention varies significantly based on the month of first purchase. By identifying high- and low-performing cohorts, the company can replicate successful acquisition and onboarding strategies while applying targeted retention interventions to weaker cohorts. This cohort-driven approach enables sustained customer loyalty and long-term revenue growth.

3.27 Q13: Analyze the lifetime value of customers acquired in different months. How can this insight inform acquisition and retention strategies?

[ ]:
```
# Acquisition Month for Each Customer
online_sales['Transaction_Date'] = pd.
 ↪to_datetime(online_sales['Transaction_Date'])

# Acquisition month = first purchase month
customer_acquisition = (
    online_sales
    .groupby('CustomerID')['Transaction_Date']
    .min()
    .reset_index()
)

customer_acquisition['Acquisition_Month'] =␣
 ↪customer_acquisition['Transaction_Date'].dt.month

customer_acquisition.head()
```

[ ]:

	CustomerID	Transaction_Date	Acquisition_Month
0	12346	2019-09-15	9
1	12347	2019-03-24	3
2	12348	2019-06-22	6

```
3       12350       2019-12-14                  12
4       12356       2019-09-15                   9
```

[ ]:
```python
# Total lifetime revenue per customer
customer_ltv = (
    online_sales
    .groupby('CustomerID')['Revenue']
    .sum()
    .reset_index()
    .rename(columns={'Revenue': 'Customer_LTV'})
)

customer_ltv.head()
```

[ ]:
```
   CustomerID  Customer_LTV
0       12346         30.99
1       12347      13834.90
2       12348       1442.12
3       12350       1360.07
4       12356       1442.47
```

[ ]:
```python
#Merge Acquisition Month with Customer LTV
ltv_acquisition = customer_acquisition.merge(
    customer_ltv,
    on='CustomerID',
    how='left'
)

ltv_acquisition.head()
```

[ ]:
```
   CustomerID Transaction_Date  Acquisition_Month  Customer_LTV
0       12346       2019-09-15                  9         30.99
1       12347       2019-03-24                  3      13834.90
2       12348       2019-06-22                  6       1442.12
3       12350       2019-12-14                 12       1360.07
4       12356       2019-09-15                  9       1442.47
```

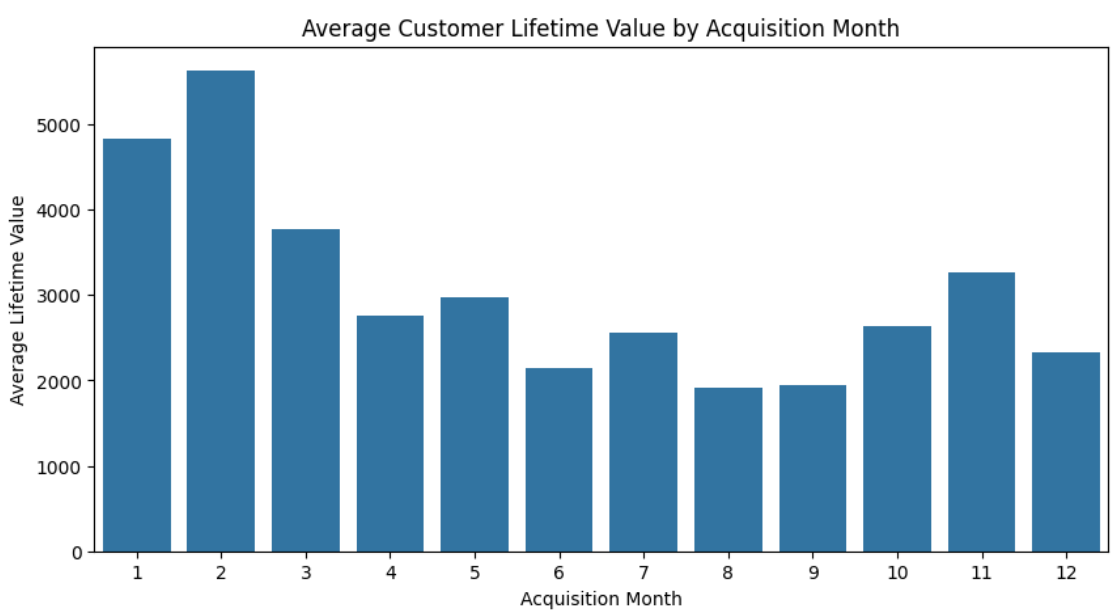
[ ]:
```python
#Average Lifetime Value by Acquisition Month
ltv_by_month = (
    ltv_acquisition
    .groupby('Acquisition_Month')['Customer_LTV']
    .mean()
    .reset_index()
    .sort_values('Customer_LTV', ascending=False)
)

ltv_by_month
```

[ ]:

	Acquisition_Month	Customer_LTV
1	2	5628.526250
0	1	4824.744465
2	3	3779.070000
10	11	3260.171029
4	5	2970.523214
3	4	2756.633497
9	10	2643.410690
6	7	2555.910000
11	12	2328.395943
5	6	2137.232190
8	9	1944.413333
7	8	1918.606444

[ ]:

```
#Visualization
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
sns.barplot(
    data=ltv_by_month,
    x='Acquisition_Month',
    y='Customer_LTV'
)

plt.title('Average Customer Lifetime Value by Acquisition Month')
plt.xlabel('Acquisition Month')
plt.ylabel('Average Lifetime Value')
plt.show()
```



3.27.1 i) Observations

- Customer lifetime value varies significantly depending on the **month of acquisition**.
- Customers acquired in certain months demonstrate **higher average lifetime revenue**, indicating stronger long-term engagement.
- Other acquisition months show **lower lifetime value**, suggesting higher churn or limited repeat purchasing.
- High acquisition volume does not always correspond to high lifetime value.

3.27.2 ii) Business Interpretation

- High-LTV acquisition months likely reflect:
  - Better customer intent
  - Effective onboarding
  - Strong product-market fit
- Low-LTV months may be driven by:
  - Short-term promotions
  - Discount-driven, low-loyalty customers
  - Seasonal or low-intent demand
- This indicates that **when customers are acquired matters as much as how many are acquired**.

3.27.3 iii) Strategic Implications for Acquisition

- Increase acquisition investment during **high-LTV months**, even if acquisition volume is lower.
- Accept **higher acquisition costs** in high-LTV periods due to stronger long-term returns.
- Re-evaluate acquisition channels used in low-LTV months.

3.27.4 iv) Strategic Implications for Retention

- Apply stronger retention programs (onboarding, loyalty, engagement) to customers acquired in **low-LTV months**.
- Introduce early repeat-purchase incentives to improve lifetime value.
- Replicate onboarding and engagement strategies from high-LTV cohorts.

3.27.5 v) Final Verdict

Customer lifetime value analysis shows that acquisition timing significantly impacts long-term revenue contribution. By prioritizing acquisition during high-LTV periods and strengthening retention efforts for low-LTV cohorts, the company can optimize marketing spend, improve customer lifetime value, and achieve more sustainable growth.

3.28 Q14: Identify seasonal trends in sales by category and location. How can the company prepare for peak and off-peak seasons to maximize revenue?

[ ]:
```
# Monthly revenue by category
category_seasonality = (
    online_sales
    .groupby(['Transaction_month', 'Product_Category'])['Revenue']
    .sum()
    .reset_index()
)

category_seasonality.head()
```

[ ]:
```
   Transaction_month Product_Category   Revenue
0                  1      Accessories     51.57
1                  1          Android     74.24
2                  1          Apparel  38300.87
3                  1        Backpacks    268.19
4                  1             Bags  10903.63
```

[ ]:
```
# Merge customer location into sales data
sales_with_location = online_sales.merge(
    customers[['CustomerID', 'Location']],
    on='CustomerID',
    how='left'
)

sales_with_location.head()
```

[ ]:
```
   CustomerID  Transaction_ID Transaction_Date     Product_SKU  \
0       17850           16679       2019-01-01  GGOENEBJ079499
1       17850           16680       2019-01-01  GGOENEBJ079499
2       17850           16681       2019-01-01  GGOEGFKQ020399
3       17850           16682       2019-01-01  GGOEGAAB010516
4       17850           16682       2019-01-01  GGOEGBJL013999

                                Product_Description Product_Category  \
0  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
1  Nest Learning Thermostat 3rd Gen-USA - Stainle…         Nest-USA
2              Google Laptop and Cell Phone Stickers           Office
3  Google Men's 100% Cotton Short Sleeve Hero Tee…          Apparel
```

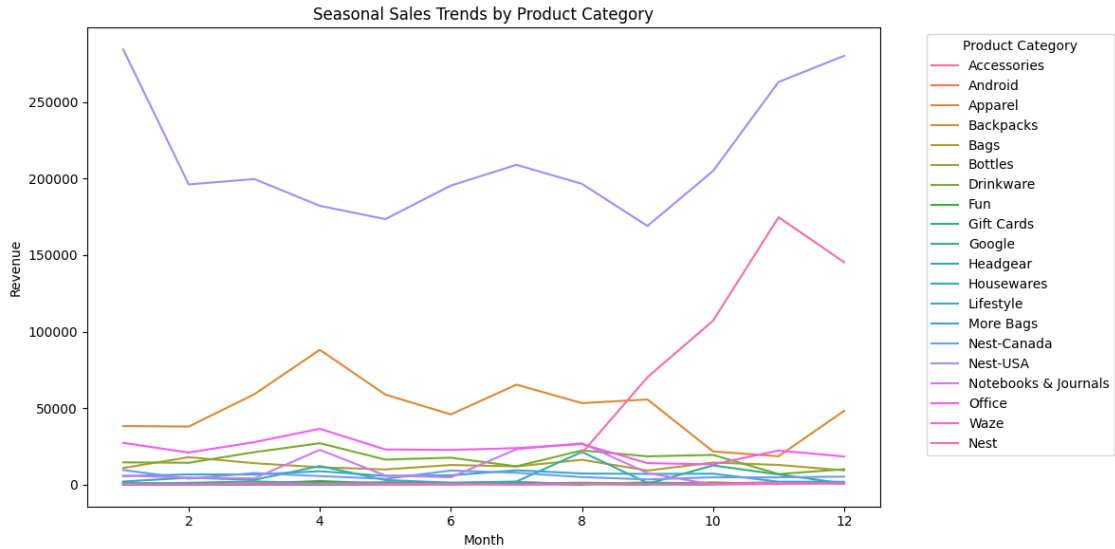
```
4                    Google Canvas Tote Natural/Navy             Bags

   Quantity  Avg_Price  Delivery_Charges Coupon_Status  Transaction_month  \
0         1     153.71               6.5          Used                  1
1         1     153.71               6.5          Used                  1
2         1       2.05               6.5          Used                  1
3         5      17.53               6.5      Not Used                  1
4         1      16.50               6.5          Used                  1

   Revenue Transaction_Month Cohort_Month  Cohort_Index Location
0   153.71           2019-01      2019-01             0  Chicago
1   153.71           2019-01      2019-01             0  Chicago
2     2.05           2019-01      2019-01             0  Chicago
3    87.65           2019-01      2019-01             0  Chicago
4    16.50           2019-01      2019-01             0  Chicago
```

[ ]:
```python
# Monthly revenue by location
location_seasonality = (
    sales_with_location
    .groupby(['Transaction_month', 'Location'])['Revenue']
    .sum()
    .reset_index()
)

location_seasonality.head()
```

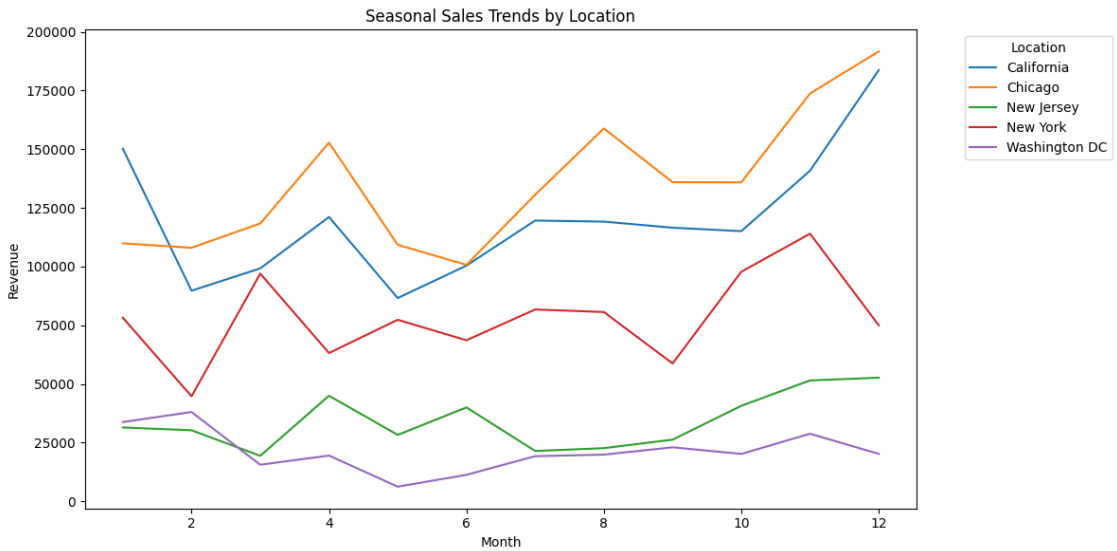
[ ]:
```
   Transaction_month       Location    Revenue
0                  1     California  150241.63
1                  1        Chicago  109911.74
2                  1     New Jersey   31475.72
3                  1       New York   78217.28
4                  1  Washington DC   33778.21
```

[ ]:
```python
#Seasonal Trends by Category
plt.figure(figsize=(12,6))
sns.lineplot(
    data=category_seasonality,
    x='Transaction_month',
    y='Revenue',
    hue='Product_Category'
)

plt.title('Seasonal Sales Trends by Product Category')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.legend(title='Product Category', bbox_to_anchor=(1.05, 1))
plt.tight_layout()
```

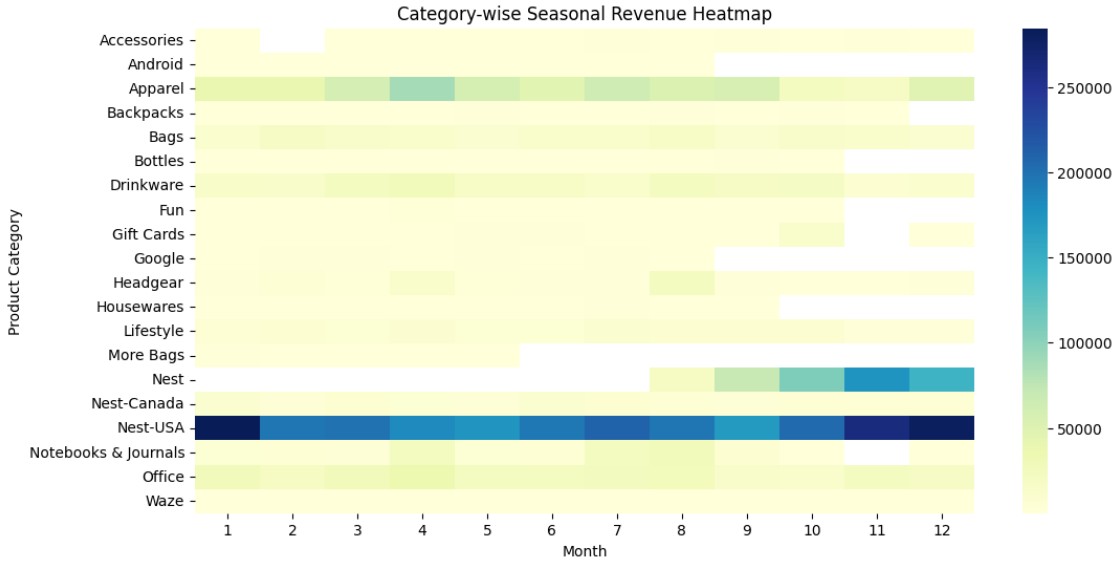
```
plt.show()
```



[ ]:
```
#Seasonal Trends by Location
plt.figure(figsize=(12,6))
sns.lineplot(
    data=location_seasonality,
    x='Transaction_month',
    y='Revenue',
    hue='Location'
)

plt.title('Seasonal Sales Trends by Location')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.legend(title='Location', bbox_to_anchor=(1.05, 1))
plt.tight_layout()
plt.show()
```



[ ]:
```
# Heatmap for category seasonality
category_heatmap = category_seasonality.pivot(
    index='Product_Category',
    columns='Transaction_month',
    values='Revenue'
)

plt.figure(figsize=(12,6))
sns.heatmap(category_heatmap, cmap='YlGnBu')
plt.title('Category-wise Seasonal Revenue Heatmap')
plt.xlabel('Month')
plt.ylabel('Product Category')
plt.show()
```



3.28.1 i) Observations

- Sales exhibit **clear seasonal patterns** across both product categories and locations.
- Certain categories show **revenue peaks during specific months**, while others remain relatively stable throughout the year.
- Locations differ in their seasonal demand patterns, indicating regional variations in purchasing behavior.
- Some months consistently emerge as **high-revenue periods (peak seasons)**, while others show noticeable declines.
- Category and location seasonality do not always align, suggesting the need for targeted planning.

3.28.2 ii) Business Interpretation

- Seasonal peaks are likely driven by:
  - Festivals and holidays
  - Climate-related demand
  - Promotional campaigns
- Categories such as electronics or lifestyle products tend to perform strongly during festive or year-end periods.
- Apparel and office-related categories show moderate seasonality with steady baseline demand.
- Location-specific trends suggest differences in customer preferences, climate, and purchasing power.

3.28.3 iii) Strategies for Peak Seasons

Inventory Management

- Increase stock levels for high-demand categories ahead of peak months.
- Prioritize inventory allocation to locations with historically higher seasonal demand.
- Use past seasonal data to forecast demand more accurately.

Promotional Planning

- Launch category-specific campaigns during known peak periods.
- Bundle complementary products to increase basket size.
- Optimize pricing strategies to capitalize on high customer intent.

3.28.4 iv) Strategies for Off-Peak Seasons

Demand Stimulation

- Introduce targeted discounts and limited-time offers.
- Promote slower-moving categories through bundling and cross-selling.
- Focus on retention campaigns and repeat-purchase incentives.

Operational Optimization

- Reduce excess inventory buildup during low-demand months.
- Use off-peak periods to test new products or campaigns.
- Shift focus toward cost-efficient marketing channels.

3.28.5 v) Final Verdict

Seasonal analysis by category and location reveals that sales performance varies significantly throughout the year. By anticipating peak and off-peak periods and aligning inventory, promotions, and regional strategies accordingly, the company can maximize revenue, reduce operational inefficiencies, and improve overall profitability.

[ ]:
```
```

3.29 Q15: Analyze daily sales trends to identify high-performing and low-performing days. What strategies can be implemented to boost sales on slower days?

[ ]:
```
#Day-Level Data
# Ensure Transaction_Date is datetime
online_sales['Transaction_Date'] = pd.
 ↪to_datetime(online_sales['Transaction_Date'])

# Extract day name
online_sales['Day_of_Week'] = online_sales['Transaction_Date'].dt.day_name()
```

```
online_sales[['Transaction_Date', 'Day_of_Week']].head()
```

[ ]:

```
  Transaction_Date Day_of_Week
0       2019-01-01     Tuesday
1       2019-01-01     Tuesday
2       2019-01-01     Tuesday
3       2019-01-01     Tuesday
4       2019-01-01     Tuesday
```

[ ]:

```
#Daily Revenue
daily_sales = (
    online_sales
    .groupby('Day_of_Week')['Revenue']
    .sum()
    .reset_index()
)

# Order days properly
day_order = [
    'Monday', 'Tuesday', 'Wednesday',
    'Thursday', 'Friday', 'Saturday', 'Sunday'
]

daily_sales['Day_of_Week'] = pd.Categorical(
    daily_sales['Day_of_Week'],
    categories=day_order,
    ordered=True
)

daily_sales = daily_sales.sort_values('Day_of_Week')
daily_sales
```

[ ]:

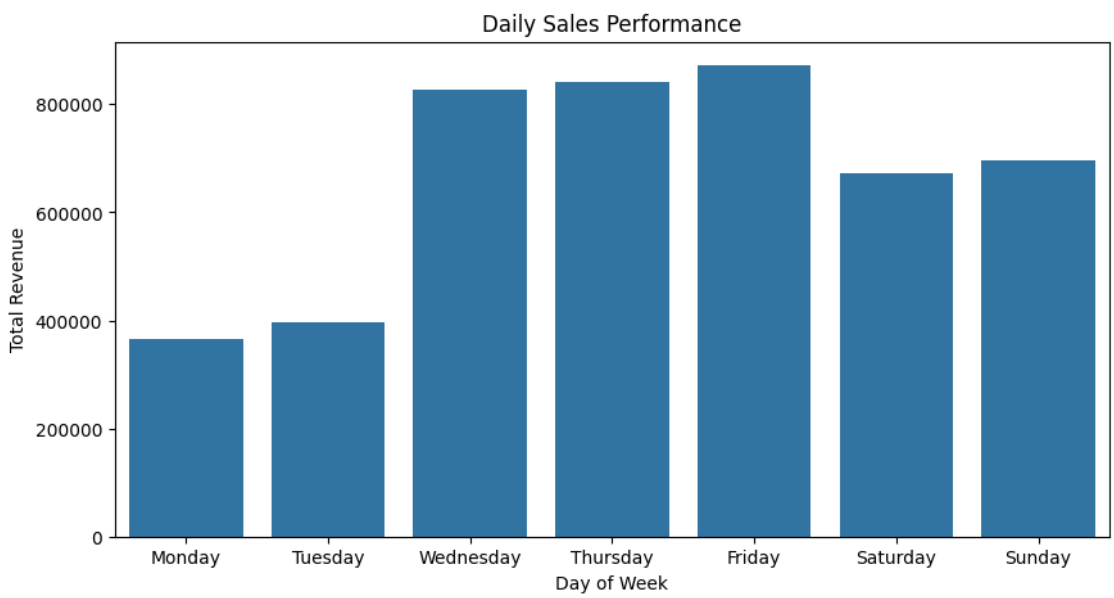
```
  Day_of_Week    Revenue
1      Monday  365626.90
5     Tuesday  396819.65
6   Wednesday  826622.00
4    Thursday  840433.85
0      Friday  872004.86
2    Saturday  673068.03
3      Sunday  696219.33
```

[ ]:

```
#High & Low Performing Days
highest_day = daily_sales.loc[daily_sales['Revenue'].idxmax()]
lowest_day = daily_sales.loc[daily_sales['Revenue'].idxmin()]

highest_day, lowest_day
```

[ ]:
```
(Day_of_Week       Friday
 Revenue        872004.86
 Name: 0, dtype: object,
 Day_of_Week      Monday
 Revenue        365626.9
 Name: 1, dtype: object)
```

[ ]:
```python
#Graph
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
sns.barplot(
    data=daily_sales,
    x='Day_of_Week',
    y='Revenue'
)

plt.title('Daily Sales Performance')
plt.xlabel('Day of Week')
plt.ylabel('Total Revenue')
plt.show()
```



3.29.1 i) Observations

- Daily sales performance varies significantly across the week.
- Certain days consistently generate **higher revenue**, indicating stronger customer activity.
- Other days show noticeably **lower sales**, suggesting reduced customer engagement.

- Weekend days generally show stronger performance compared to weekdays.
- Sales tend to dip during mid-week or early-week periods.

3.29.2 ii) Business Interpretation

- High-performing days likely align with:
  - Increased customer free time
  - Promotional timing
  - Higher online shopping intent
- Low-performing days may reflect:
  - Workday constraints
  - Lower promotional visibility
  - Reduced urgency to purchase
- Customer purchase behavior follows **predictable weekly patterns**.

3.29.3 iii) Strategies to Boost Sales on Slower Days

3.29.4 Targeted Promotions

- Introduce **weekday-exclusive discounts** on low-performing days.
- Run flash sales or limited-time offers mid-week.

Marketing Optimization

- Schedule email and push campaigns on slower days.
- Use reminder notifications for abandoned carts.

Incentives & Engagement

- Offer free shipping or bonus rewards on low-traffic days.
- Promote bundles to increase basket size.

Operational Alignment

- Launch new product releases or campaigns on slower days to spread demand evenly.
- Shift some marketing budget from peak days to underperforming days.

3.29.5 iv) Final Verdict

Daily sales analysis reveals clear patterns in customer purchasing behavior across the week. By leveraging high-performing days for revenue maximization and applying targeted promotions and engagement strategies on slower days, the company can smooth demand, increase overall sales, and improve operational efficiency.

[ ]:
```
!pip install jupyter nbconvert
```

```
Requirement already satisfied: jupyter in /usr/local/lib/python3.12/dist-
packages (1.1.1)
Requirement already satisfied: nbconvert in /usr/local/lib/python3.12/dist-
packages (7.16.6)
Requirement already satisfied: notebook in /usr/local/lib/python3.12/dist-
packages (from jupyter) (6.5.7)
Requirement already satisfied: jupyter-console in
/usr/local/lib/python3.12/dist-packages (from jupyter) (6.6.3)
Requirement already satisfied: ipykernel in /usr/local/lib/python3.12/dist-
packages (from jupyter) (6.17.1)
Requirement already satisfied: ipywidgets in /usr/local/lib/python3.12/dist-
packages (from jupyter) (7.7.1)
Requirement already satisfied: jupyterlab in /usr/local/lib/python3.12/dist-
packages (from jupyter) (4.5.2)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (4.13.5)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-
packages (from bleach[css]!=5.0.0->nbconvert) (6.3.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (5.9.1)
Requirement already satisfied: jupyterlab-pygments in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (3.0.3)
Requirement already satisfied: mistune<4,>=2.0.3 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (3.2.0)
Requirement already satisfied: nbclient>=0.5.0 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (0.10.4)
Requirement already satisfied: nbformat>=5.7 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (25.0)
Requirement already satisfied: pandocfilters>=1.4.1 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (1.5.1)
Requirement already satisfied: pygments>=2.4.1 in
/usr/local/lib/python3.12/dist-packages (from nbconvert) (2.19.2)
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-
packages (from nbconvert) (5.7.1)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-
packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert)
(1.4.0)
Requirement already satisfied: platformdirs>=2.5 in
/usr/local/lib/python3.12/dist-packages (from jupyter-core>=4.7->nbconvert)
(4.5.1)
Requirement already satisfied: jupyter-client>=6.1.12 in
/usr/local/lib/python3.12/dist-packages (from nbclient>=0.5.0->nbconvert)
(7.4.9)
Requirement already satisfied: fastjsonschema>=2.15 in
/usr/local/lib/python3.12/dist-packages (from nbformat>=5.7->nbconvert) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in
/usr/local/lib/python3.12/dist-packages (from nbformat>=5.7->nbconvert) (4.26.0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-
packages (from beautifulsoup4->nbconvert) (2.8.1)
Requirement already satisfied: typing-extensions>=4.0.0 in
/usr/local/lib/python3.12/dist-packages (from beautifulsoup4->nbconvert)
(4.15.0)
Requirement already satisfied: debugpy>=1.0 in /usr/local/lib/python3.12/dist-
packages (from ipykernel->jupyter) (1.8.15)
Requirement already satisfied: ipython>=7.23.1 in
/usr/local/lib/python3.12/dist-packages (from ipykernel->jupyter) (7.34.0)
Requirement already satisfied: matplotlib-inline>=0.1 in
/usr/local/lib/python3.12/dist-packages (from ipykernel->jupyter) (0.2.1)
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.12/dist-
packages (from ipykernel->jupyter) (1.6.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages
(from ipykernel->jupyter) (5.9.5)
Requirement already satisfied: pyzmq>=17 in /usr/local/lib/python3.12/dist-
packages (from ipykernel->jupyter) (26.2.1)
Requirement already satisfied: tornado>=6.1 in /usr/local/lib/python3.12/dist-
packages (from ipykernel->jupyter) (6.5.1)
Requirement already satisfied: ipython-genutils~=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from ipywidgets->jupyter) (0.2.0)
Requirement already satisfied: widgetsnbextension~=3.6.0 in
/usr/local/lib/python3.12/dist-packages (from ipywidgets->jupyter) (3.6.10)
Requirement already satisfied: jupyterlab-widgets>=1.0.0 in
/usr/local/lib/python3.12/dist-packages (from ipywidgets->jupyter) (3.0.16)
Requirement already satisfied: prompt-toolkit>=3.0.30 in
/usr/local/lib/python3.12/dist-packages (from jupyter-console->jupyter) (3.0.52)
Requirement already satisfied: async-lru>=1.0.0 in
/usr/local/lib/python3.12/dist-packages (from jupyterlab->jupyter) (2.0.5)
Requirement already satisfied: httpx<1,>=0.25.0 in
/usr/local/lib/python3.12/dist-packages (from jupyterlab->jupyter) (0.28.1)
Requirement already satisfied: jupyter-lsp>=2.0.0 in
/usr/local/lib/python3.12/dist-packages (from jupyterlab->jupyter) (2.3.0)
Requirement already satisfied: jupyter-server<3,>=2.4.0 in
/usr/local/lib/python3.12/dist-packages (from jupyterlab->jupyter) (2.14.0)
Requirement already satisfied: jupyterlab-server<3,>=2.28.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from jupyterlab->jupyter) (2.28.0)
Requirement already satisfied: notebook-shim>=0.2 in
/usr/local/lib/python3.12/dist-packages (from jupyterlab->jupyter) (0.2.4)
Requirement already satisfied: setuptools>=41.1.0 in
/usr/local/lib/python3.12/dist-packages (from jupyterlab->jupyter) (75.2.0)
Requirement already satisfied: argon2-cffi in /usr/local/lib/python3.12/dist-
packages (from notebook->jupyter) (25.1.0)
Requirement already satisfied: Send2Trash>=1.8.0 in
/usr/local/lib/python3.12/dist-packages (from notebook->jupyter) (2.0.0)
Requirement already satisfied: terminado>=0.8.3 in
/usr/local/lib/python3.12/dist-packages (from notebook->jupyter) (0.18.1)
Requirement already satisfied: prometheus-client in
/usr/local/lib/python3.12/dist-packages (from notebook->jupyter) (0.23.1)
Requirement already satisfied: nbclassic>=0.4.7 in
/usr/local/lib/python3.12/dist-packages (from notebook->jupyter) (1.3.3)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages
(from httpx<1,>=0.25.0->jupyterlab->jupyter) (4.12.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-
packages (from httpx<1,>=0.25.0->jupyterlab->jupyter) (2026.1.4)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-
packages (from httpx<1,>=0.25.0->jupyterlab->jupyter) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages
(from httpx<1,>=0.25.0->jupyterlab->jupyter) (3.11)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-
packages (from httpcore==1.*->httpx<1,>=0.25.0->jupyterlab->jupyter) (0.16.0)
Requirement already satisfied: jedi>=0.16 in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->jupyter) (0.19.2)
Requirement already satisfied: decorator in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->jupyter) (4.4.2)
Requirement already satisfied: pickleshare in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->jupyter) (0.7.5)
Requirement already satisfied: backcall in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->jupyter) (0.2.0)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->jupyter) (4.9.0)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-
packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (25.4.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/usr/local/lib/python3.12/dist-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in
/usr/local/lib/python3.12/dist-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in
/usr/local/lib/python3.12/dist-packages (from
jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.30.0)
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-
packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (0.4)
```

```
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: jupyter-events>=0.9.0 in
/usr/local/lib/python3.12/dist-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (0.12.0)
Requirement already satisfied: jupyter-server-terminals>=0.4.4 in
/usr/local/lib/python3.12/dist-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (0.5.3)
Requirement already satisfied: overrides>=5.0 in /usr/local/lib/python3.12/dist-
packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (7.7.0)
Requirement already satisfied: websocket-client>=1.7 in
/usr/local/lib/python3.12/dist-packages (from jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (1.9.0)
Requirement already satisfied: argon2-cffi-bindings in
/usr/local/lib/python3.12/dist-packages (from argon2-cffi->notebook->jupyter)
(25.1.0)
Requirement already satisfied: babel>=2.10 in /usr/local/lib/python3.12/dist-
packages (from jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter) (2.17.0)
Requirement already satisfied: json5>=0.9.0 in /usr/local/lib/python3.12/dist-
packages (from jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter) (0.13.0)
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.12/dist-
packages (from jupyterlab-server<3,>=2.28.0->jupyterlab->jupyter) (2.32.4)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-
packages (from prompt-toolkit>=3.0.30->jupyter-console->jupyter) (0.2.14)
Requirement already satisfied: ptyprocess in /usr/local/lib/python3.12/dist-
packages (from terminado>=0.8.3->notebook->jupyter) (0.7.0)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
/usr/local/lib/python3.12/dist-packages (from
jedi>=0.16->ipython>=7.23.1->ipykernel->jupyter) (0.8.5)
Requirement already satisfied: python-json-logger>=2.0.4 in
/usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (4.0.0)
Requirement already satisfied: pyyaml>=5.3 in /usr/local/lib/python3.12/dist-
packages (from jupyter-events>=0.9.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (6.0.3)
Requirement already satisfied: rfc3339-validator in
/usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (0.1.4)
Requirement already satisfied: rfc3986-validator>=0.1.1 in
/usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (0.1.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.8.2->jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.31->jupyterlab-
server<3,>=2.28.0->jupyterlab->jupyter) (3.4.4)
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.31->jupyterlab-
server<3,>=2.28.0->jupyterlab->jupyter) (2.5.0)
Requirement already satisfied: cffi>=1.0.1 in /usr/local/lib/python3.12/dist-
packages (from argon2-cffi-bindings->argon2-cffi->notebook->jupyter) (2.0.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-
packages (from cffi>=1.0.1->argon2-cffi-
bindings->argon2-cffi->notebook->jupyter) (2.23)
Requirement already satisfied: fqdn in /usr/local/lib/python3.12/dist-packages
(from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (1.5.1)
Requirement already satisfied: isoduration in /usr/local/lib/python3.12/dist-
packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.9.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (20.11.0)
Requirement already satisfied: jsonpointer>1.13 in
/usr/local/lib/python3.12/dist-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (3.0.0)
Requirement already satisfied: rfc3987-syntax>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (1.1.0)
Requirement already satisfied: uri-template in /usr/local/lib/python3.12/dist-
packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.9.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.3.0)
Requirement already satisfied: webcolors>=24.6.0 in
/usr/local/lib/python3.12/dist-packages (from jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (25.10.0)
Requirement already satisfied: lark>=1.2.2 in /usr/local/lib/python3.12/dist-
packages (from rfc3987-syntax>=1.1.0->jsonschema[format-
nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-
server<3,>=2.4.0->jupyterlab->jupyter) (1.3.1)
Requirement already satisfied: arrow>=0.15.0 in /usr/local/lib/python3.12/dist-
packages (from isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.9.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (1.4.0)
Requirement already satisfied: tzdata in /usr/local/lib/python3.12/dist-packages
(from arrow>=0.15.0->isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-
events>=0.9.0->jupyter-server<3,>=2.4.0->jupyterlab->jupyter) (2025.3)
```

[ ]:
```
!apt-get update
!apt-get install -y texlive-xetex texlive-fonts-recommended␣
↪texlive-plain-generic
```

```
 0% [Working]              Get:1 https://cli.github.com/packages stable
InRelease [3,917 B]
Hit:2 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Hit:3 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
```

```
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:5 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:9 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Fetched 3,917 B in 1s (3,112 B/s)
Reading package lists… Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists… Done
Building dependency tree… Done
Reading state information… Done
texlive-fonts-recommended is already the newest version (2021.20220204-1).
texlive-plain-generic is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 42 not upgraded.
```

[ ]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

[ ]:
```
!find /content/drive -name "*.ipynb"
```

[ ]:
```
!jupyter nbconvert --to pdf "/content/01_Case study: E-commerce Marketing and␣
↪sales.ipynb"
```

```
[NbConvertApp] WARNING | pattern '/content/01_Case study: E-commerce Marketing
and sales.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
```

```
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and
include the error message in the cell output (the default behaviour is to abort
conversion). This flag is only relevant if '--execute' was specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with
default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--ClearOutputPreprocessor.enabled=True]
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each
cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False
--NbConvertApp.export_format=notebook --FilesWriter.build_directory=
--CoalesceStreamsPreprocessor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True
--TemplateExporter.exclude_output_prompt=True]
--no-input
```

```
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True
--TemplateExporter.exclude_input=True
--TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the
system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
    Embed the images as base64 dataurls in the output. This flag is only useful
for the HTML/WebPDF/Slides exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR',
'CRITICAL']
    Default: 30
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
            ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook',
'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
            or a dotted object name that represents the import path for an
            ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
```

```
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
    as prebuilt extension for the lab template)
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                        results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
                                        results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
                Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
                                  to output to the directory of each notebook.
To recover
                                  previous default behaviour (outputting to the
current
                                  working directory) use . as the flag value.
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
            This defaults to the reveal CDN, but can be any url pointing to a
copy
            of reveal.js.
            For speaker notes to work, this must be a relative path to a local
            copy of reveal.js: e.g., "reveal.js".
            If a relative path is given, it must be a subdirectory of the
            current directory (from which the server is run).
            See the usage documentation
            (https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-
html-slideshow)
            for more details.
    Default: ''
```

```
    Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
    The nbformat version to write.
            Use this to downgrade notebooks.
    Choices: any of [1, 2, 3, 4]
    Default: 4
    Equivalent to: [--NotebookExporter.nbformat_version]

Examples
--------

    The simplest way to use nbconvert is

            > jupyter nbconvert mynotebook.ipynb --to html

            Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown',
'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides',
'webpdf'].

            > jupyter nbconvert --to latex mynotebook.ipynb

            Both HTML and LaTeX support multiple output templates. LaTeX
includes
            'base', 'article' and 'report'.  HTML includes 'basic', 'lab' and
            'classic'. You can specify the flavor of the format used.

            > jupyter nbconvert --to html --template lab mynotebook.ipynb

            You can also pipe the output to stdout, rather than a file

            > jupyter nbconvert mynotebook.ipynb --stdout

            PDF is generated via latex

            > jupyter nbconvert mynotebook.ipynb --to pdf

            You can get (and serve) a Reveal.js-powered slideshow

            > jupyter nbconvert myslides.ipynb --to slides --post serve

            Multiple notebooks can be given at the command line in a couple of
            different ways:

            > jupyter nbconvert notebook*.ipynb
            > jupyter nbconvert notebook1.ipynb notebook2.ipynb

            or you can specify the notebooks list in a config file, containing::
```

```
                c.NbConvertApp.notebooks = ["my_notebook.ipynb"]

            > jupyter nbconvert --config mycfg.py

To see all available configurables, use `--help-all`.
```

[ ]:
```
!jupyter nbconvert --to pdf "/content/drive/MyDrive/Colab Notebooks/Business␣
↪Case study/01/01_Case study: E-commerce Marketing and sales.ipynb"
```

[ ]:
```

```